UPDATE

MCSL 381 release

PluginHive MCSL · Shopify / WooCommerce / BigCommerce / Magento / PrestaShop

| Story / card | Platform | Carrier scope | Toggle / prerequisite signal |

Generated June 2026 · PluginHive QA Team

Included Story Cards

Story / card	Platform	Carrier scope	Toggle / prerequisite signal
ZI-544	Shopify, Magento	UPS, Amazon Shipping	large.batch.fast.path.enabled, labelbatch.shopify.tag.sync.enabled, batch.snapshot.target_active_batch_only
ZI-538	Shopify	FedEx, Australia Post, MyPost	account-uuid.australiaPost.recipient.contact.required.enabled, australiaPost.recipient.contact.required.enabled
ZI-540	Shopify, WooCommerce, BigCommerce, Magento, PrestaShop	NZ Post	off, on
ZI-531	Shopify, WooCommerce, BigCommerce, Magento, PrestaShop	DHL, EasyPost, EasyPost USPS	None detected
ZI-552	Shopify, WooCommerce, BigCommerce, Magento, PrestaShop	Carrier-neutral	tracking.email.suppress.no.smtp.enabled, tracking.email.suppress.no.smtp.disabled, feature-flag, log-only
ZI-530	Shopify, WooCommerce, BigCommerce, Magento, PrestaShop	FedEx, Australia Post	None detected
ZI-524	Shopify, WooCommerce, BigCommerce, Magento, PrestaShop	Amazon Shipping	account_uuid.eta.column.enabled
ZI-523	Shopify, WooCommerce, BigCommerce, Magento, PrestaShop	FedEx, UPS, DHL, USPS, Canada Post, Australia Post, Amazon Shipping	None detected
ZI-555	Shopify	Carrier-neutral	None detected
ZI-520	Shopify	Carrier-neutral	None detected
Carrier Integration India Post	Shopify, WooCommerce, BigCommerce, Magento, PrestaShop	FedEx, Australia Post, NZ Post	flag and cross-carrier regression coverage required by the AC skill

Story / card	Platform	Carrier scope	Toggle / prerequisite signal
Carrier Integration DHL Express	Shopify, WooCommerce, BigCommerce, Magento, PrestaShop	DHL	None detected
WSS: Add flat rate adjustment shown when carrier rates fail (ZD #394137)	WooCommerce	UPS	None detected

How Support Should Use This Package

› Use each card section as the support/demo guide for that feature.

› Confirm customer/test platform, live store, carrier account, order/product, and toggle state before promising behavior.

› Capture the escalation packet listed in the relevant card section if behavior does not match.

ZI-544 - From SL: ZI-544 — Love of India: large-batch UX, retry-state, print mismatch, cascade routing (consolidates ZI-545, ZI-546, ZI-547) [#390510]

Support Guide: ZI-544 — Large-Batch UX, Retry-State Fix, and Batch Tag Sync

From SL: ZI-544 — Love of India: large-batch UX, retry-state, print mismatch, cascade routing (consolidates ZI-545, ZI-546, ZI-547) [#390510]

Trello card: https://trello.com/c/qaq1crm5/4455

Feature Summary

This card delivers two bug fixes and one new feature to the MCSL Label Batch workflow, primarily affecting Shopify (the test platform). The first fix resolves a UX regression where large batches (1,000+ orders) triggered a premature redirect or displayed a false "Error in Batch Creation" banner before processing had completed. The second fix corrects retry-state display: failed orders in an original batch were incorrectly updating their status and carrier name when a retry batch succeeded — they should remain frozen at their original failed state, with the retry result visible only in the retry batch. The new feature adds the ability to add and remove Shopify order tags directly from the Label Batch page, with tag changes syncing back to Shopify orders and MCSL order records independently of label or manifest status. Three feature flags gate this behaviour. Note: the print mismatch (fulfilled "Not in Ship" orders) and cascade priority-based carrier routing (Amazon / Delhivery / XpressBees) described in the original card title were **not implemented** in this release.

Toggles & Prerequisites

Toggle / config note	What support should confirm
`{accountUUID}.large.batch.fast.path.enabled` = `true`	Must be ON for large-batch fast-path UX fix (no premature redirect, no false error banner). Confirm in account config before troubleshooting batch redirect issues.

Toggle / config note	What support should confirm
`{accountUUID}.batch.snapshot.target_active_batch_only = true`	Must be ON alongside `large.batch.fast.path.enabled` for correct batch progress scoping. Confirm both are enabled together.
`{accountUUID}.labelbatch.shopify.tag.sync.enabled = true`	Must be ON for the Add/Remove Tags feature on the Label Batch page. Without this, the tag UI may not appear or tag changes will not sync to Shopify.
Release prerequisite	MCSL 381 (Iteration backlog). Confirm the store is on build 381 or later before investigating these behaviours.
Carrier account prerequisite	UPS and Amazon Shipping are the carriers referenced in the card context. Confirm carrier accounts are connected under hamburger menu > Carriers if batch failures are carrier-related.
Customer / test platform	Shopify (test platform). Steps below are written for Shopify.
Shopify Admin tag write permission	For tag sync to work, the MCSL app must have write access to Shopify orders. Confirm OAuth scopes include `write_orders` if tags are not appearing in Shopify.

Step-by-Step Support Walkthrough

Part A — Large-Batch UX (1,000+ Orders, No Premature Redirect)

1. **Confirm toggles are active.** Before any testing, verify that both `large.batch.fast.path.enabled` and `batch.snapshot.target_active_batch_only` are set to `true` for the merchant's account UUID. If either is missing or false, the fast-path UX will not apply and the old redirect behaviour may still occur.

2. **Navigate to the MCSL Orders tab** in Shopify Admin > Apps > PH Multi Carrier Shipping Label App > **ORDERS**.

3. **Select 100+ orders** using the bulk-select checkbox. For full regression coverage, test with 1,000 and 2,000 orders if the merchant's store volume allows.

4. **Trigger bulk label generation.** Click the bulk label generation action (Create Batch / Generate Label button). Confirm the following immediately after clicking:

› The user is redirected to the **Label Batch page** (not away from it).

› No `batchCreationFailed` error banner appears on screen during active processing.

› The batch progress table remains visible and mounted throughout the entire processing window.

› Each batch group row updates its status (e.g., Processing → Completed or Failed) without a full-page redirect.

5. **Simulate a browser refresh mid-batch.** Refresh the browser while the batch is still processing. Confirm:

› The batch continues processing after the refresh.

› The progress indicator resumes correctly.

› No duplicate batch is created.

6. **Verify final batch counts.** Once processing completes, confirm on the Label Batch page:

› Batch count matches the number of selected orders.

› Labels generated count matches the success count.

› Failed count matches actual failures.

› **Request/log fields to verify:** Check the MCSL request log (hamburger menu > Settings > Request Log) for any `batchCreationFailed` events or unexpected redirect triggers during the processing window. Confirm no error events are logged while batch items still show in-progress status.

Part B — Retry-State: Failed Orders in Original vs. Retry Batch

7. **Identify a batch with failed orders.** In the MCSL **ORDERS** tab > Label Batch page, locate a batch that contains at least one failed label generation attempt.

8. **Verify the original batch state is frozen.** In the original batch:

› Failed orders must remain in **Failed** status. Status must NOT change from Failed → Success even after a retry succeeds.

› The **carrier name** displayed for failed orders must remain the original carrier (e.g., UPS or Amazon Shipping) — it must not update to the retry carrier.

9. **Trigger a retry.** Use the retry action on the failed orders. Confirm a **new, separate retry batch** is created.

10. **Verify the retry batch independently.** In the retry batch:

› Failed orders from the original batch appear here.

› The retry carrier name is shown only in this retry batch, not in the original.

› If the retry succeeds, the order status updates in the retry batch.

› If the retry fails again, it can be retried a further time (creating another separate batch).

11. **Verify print isolation.** When printing from the original batch, confirm only original-batch labels are included. When printing from the retry batch, confirm only retry labels are included. Tracking numbers must remain associated with the correct batch.

12. **Verify order grid and order summary.** Navigate to the MCSL **ORDERS** tab and open an affected order's summary page. Confirm the label status reflects the correct batch outcome and is not incorrectly overwritten by a retry result.

› **Request/log fields to verify:** In the request log, confirm that the status update payload for the original batch does not include a `SUCCESS` status write for orders that failed in that batch. The retry batch should carry its own status update independently.

Part C — Add/Remove Tags on Label Batch Page

13. **Confirm** `labelbatch.shopify.tag.sync.enabled` **is** `true` for the account UUID before testing tag functionality.

14. **Navigate to the Label Batch page** in MCSL (ORDERS tab > Label Batch). Select a batch.

15. **Add a tag.** Use the tag add control on the batch detail page. Add one or more tags (including a tag with special characters to verify handling). Confirm:

› Tags appear in the batch details page immediately.

› Tags persist after a page refresh in the app.

› Tags are reflected in the MCSL **ORDERS** tab for the orders in that batch (note: tag update in the order grid occurs after the Shopify order update is confirmed successful — there may be a short sync delay).

16. **Verify Shopify-side tag sync.** In Shopify Admin > Orders, open one of the orders in the batch. Confirm the added tag(s) appear on the Shopify order record.

› **Request/log fields to verify:** In the MCSL request log, confirm the Shopify order update request includes the correct `tags` field with the newly added tag(s). Verify the response confirms a successful write.

17. **Remove a tag.** Use the tag remove control on the same batch. Confirm:

› The tag is removed from the batch details page.

› The tag is removed from MCSL order records.

› The tag is removed from the corresponding Shopify order(s) in Shopify Admin > Orders.

18. **Test tag operations independent of label/manifest status.** Confirm tags can be added and removed regardless of whether the batch label status is Success, Failed, or Processing, and regardless of manifest status (Success, Failed, or Pending).

19. **Test multi-tag operations.** Add multiple tags in a single action. Remove multiple tags in a single action. Confirm no duplicate tags are created when adding a tag that already exists on the order.

20. **Test tag filtering in the MCSL Orders tab.** After adding a tag, use the tag filter in the MCSL **ORDERS** tab to filter by the newly added tag. Confirm correct orders are returned.

> □□ **Known limitation (QA-noted):** The MCSL Orders grid currently supports filtering by **one tag at a time**. Multi-tag filtering from the batch page is not yet fully supported in the order grid. Do not escalate this as a bug — it is a documented known gap from QA.

> □□ **Known QA note on tag removal count:** If a batch with no existing tags is included in a multi-batch tag removal selection, the removal note may display an inflated count (e.g., "Removing 6 of 6 tags" instead of "Removing 4 of 4 tags"). This is a known cosmetic issue — confirm from live store/card context whether a fix is scheduled.

21. **Verify tag isolation.** Confirm that tag updates apply only to orders belonging to the selected batch, not to orders in other batches.

Expected Behaviour

What support should

ZI-538 - From SL: ZI-538 — MCSL: sanitise null AusPost recipient phone by 2026-08-04 [#390097]

Support Guide: ZI-538 / MCSL 381 — Sanitise Null AusPost Recipient Phone

From SL: ZI-538 — MCSL: sanitise null AusPost recipient phone by 2026-08-04 [#390097]

Trello card: https://trello.com/c/br0i86eK/4457

Feature Summary

Australia Post will enforce a hard contact-required policy from **2026-08-04**, rejecting eParcel and MyPost Business (MPB) label and manifest requests where the recipient has neither a phone number nor an email address. This change adds a pre-flight validator in MCSL that blocks label and manifest generation for AusPost orders missing both contact fields, surfaces a clear error in the Order Grid and Order Summary page, and writes a matching error entry to the request log. When the toggle is enabled in **strict mode**, generation is fully blocked and no API call is made to AusPost. A **warn mode** exists that shows a non-blocking advisory without stopping the label. Merchants can backfill a phone number directly in the MCSL in-app override panel to unblock an order without modifying the original Shopify source order. The validator is scoped to Australia Post (eParcel and MPB) only and does not affect FedEx or other carriers.

Toggles & Prerequisites

Toggle / config note	What support should confirm
`account-uuid.australiaPost.recipient.contact.required.enabled` (account-level)	Confirm this is set to `true` for the specific merchant's account UUID to enable strict mode for that store only
`australiaPost.recipient.contact.required.enabled` (global/platform-level)	Confirm whether the global toggle is enabled; account-level toggle takes precedence when set
Strict mode vs. warn mode	Confirm which mode is active — strict mode blocks generation entirely; warn mode shows a non-blocking advisory and still submits the request
Toggle `false` / absent	Legacy behaviour is preserved — no pre-flight block, no new warning, AusPost request is sent using the existing payload
Australia Post carrier account configured	Confirm the store has an active eParcel or MyPost Business carrier account connected under hamburger menu > Carriers
MCSL release prerequisite	Feature is part of MCSL 381 (SL iteration backlog); confirm the store is on MCSL 381 or later
Test/customer platform	Shopify — validated on Shopify Admin > Apps > PH Multi Carrier Shipping Label App
AusPost policy enforcement date	2026-08-04 — warn mode is intended as a pre-deadline advisory; strict mode enforces the block immediately when toggled on

---

Step-by-Step Support Walkthrough

Verify the toggle state before any other step.

1. **Confirm the toggle is enabled for the merchant's account.**

Navigate to the internal config store or account settings and check for:

› `account-uuid.australiaPost.recipient.contact.required.enabled: true` (account-level, takes precedence)

› `australiaPost.recipient.contact.required.enabled: true` (global fallback)

If neither is set or both are `false`, the validator is inactive and legacy behaviour applies — stop here and confirm with the merchant whether strict mode should be on.

2. Identify a test order with a missing recipient phone and email.

In **Shopify Admin > Orders**, locate an order where the customer record has no phone number and no email, or where the phone field is whitespace-only. Note the order number for cross-referencing in MCSL.

3. Open the order in the MCSL Order Grid.

In **Shopify Admin > Apps > PH Multi Carrier Shipping Label App**, go to the **ORDERS** tab. Locate the test order. Confirm the order row shows a pre-flight warning indicator if the order is AusPost-eligible and the contact fields are missing.

4. Attempt single label generation (strict mode).

Click **Generate Label** for the order. The expected result is that label generation is blocked immediately.

› Confirm the **Order Grid** displays the error: *"Recipient phone or email is required by Australia Post from 2026-08-04"*

› Open the order to the **Order Summary / Prepare Shipment** page and confirm the same error message is visible there.

› **Request/log fields to verify:** Open hamburger menu > **Settings > Request Log** (or the equivalent request log path for this store). Confirm a log entry exists for this attempt and contains the correct error message. Confirm **no outbound AusPost API request** was recorded — the block should prevent any payload being sent.

5. Attempt single label generation (warn mode, if applicable).

If the toggle is in warn mode rather than strict mode, repeat step 4. The label should generate successfully. Confirm:

› A non-blocking warning is shown in the UI: *"Will be blocked by AusPost after 2026-08-04"*

› The same warning indicator is visible on the Order Grid row.

› The AusPost API request is sent and a tracking number is returned.

› **Request/log fields to verify:** Confirm the request log shows an outbound AusPost payload and a successful response with a tracking number.

6. Verify the backfill flow unblocks a blocked order.

With the order in its blocked state (from step 4), open the **in-app override panel** for that order (within the MCSL Order Summary / Prepare Shipment view on Shopify). Enter a valid recipient phone number and click **Save**.

› Confirm the backfilled phone is persisted on the MCSL order copy.

› Open **Shopify Admin > Orders** and confirm the source order is **unchanged** — the backfill must not write back to Shopify.

› Re-run **Generate Label**. Confirm the validator passes and the label is generated successfully.

› **Request nodes to verify:** In the request log, confirm the AusPost JSON payload contains the backfilled phone number in the recipient contact node.

7. **Verify bulk label generation partially blocks invalid orders.**

On the **ORDERS** tab, select a mixed set of orders (some with valid contact, some without). Click **Generate Labels** (bulk). Confirm:

› Labels are generated for orders with valid recipient contact.

› Orders missing both phone and email are skipped with the reason *"Missing recipient phone/email"*.

› No AusPost API call is recorded in the request log for the skipped orders.

› The bulk result summary lists the skipped order numbers.

8. **Verify the validator does not affect FedEx orders.**

On the **ORDERS** tab, locate a FedEx-eligible order. Click **Generate Label**. Confirm:

› Label generation is **not** blocked by the AusPost validator.

› The FedEx flow proceeds normally using its existing recipient-phone fallback.

› **Request/log fields to verify:** Confirm the request log shows a FedEx outbound request with no AusPost validator error.

9. **Verify the AusPost chunked manifest flow (if applicable).**

If the merchant uses the AusPost manifest/chunked manifest flow, run a manifest batch that includes contact-invalid orders. Confirm:

› Contact-invalid orders are excluded from the manifest chunk before submission.

› The manifest is submitted with only valid orders.

› Excluded orders are surfaced in the result summary with the same missing-contact error.

› **Request/log fields to verify:** Confirm the request log shows no AusPost manifest API call for the excluded orders.

10. **Verify whitespace-only phone is treated as missing.**

Locate or create a test order where the recipient phone field contains only spaces. Attempt label generation. Confirm the validator treats this as a missing phone and blocks generation with the same error as a fully null phone field.

Expected Behaviour

What support should observe	How to confirm
Order Grid shows error message when label generation is attempted for an AusPost order with no recipient phone and no email (strict mode)	Error text *"Recipient phone or email is required by Australia Post from 2026-08-04"* visible on the order row after generation attempt
Order Summary / Prepare Shipment page shows the same error message	Open the order in MCSL; error is displayed on the order detail/summary view
No outbound AusPost API request is made when the validator blocks (strict mode)	Request log (hamburger menu > Settings > Request Log) shows the error entry but no AusPost API request payload for that order

What support should observe	How to confirm
Request log contains the correct error message text	Log entry for the blocked attempt shows *"Recipient phone or email is required by Australia Post from 2026-08-04"* or equivalent
Warn mode shows a non-blocking advisory and still generates the label	Warning indicator visible on Order Grid row and Order Summary; label and tracking number returned; AusPost request visible in log
Backfilled phone is reflected in the AusPost JSON payload	Request log for the re-attempted label shows recipient contact node populated with the backfilled phone number
Shopify source order is not modified by the in-app backfill	Shopify Admin > Orders shows the original order data unchanged after backfill is saved in MCSL
Bulk generation skips invalid orders and reports them	Bulk result summary lists skipped order numbers with *"Missing recipient phone/email"* reason; no log entries for skipped orders
FedEx label generation is unaffected by the AusPost validator	FedEx label generates normally; no AusPost validator error in the FedEx request log entry
Whitespace-only phone is treated as missing	Validator blocks generation for whitespace-only phone with the same error as a null phone
Toggle `false` preserves legacy behaviour	No pre-flight block, no new warning, AusPost request sent as before; no validator error in log
MyPost Business (MPB) adapter blocks the same way as eParcel	Same error message and same log behaviour observed for MPB orders as for eParcel orders

---

Merchant-Safe Explanation

Australia Post is introducing a mandatory requirement from **4 August 2026** that every shipment must include the recipient's phone number or

ZI-540 - From SL: ZI-540 — NZ Post Enhancements: SOv2 + Label upgrades + Scoping (consolidates ZI-541, ZI-543) [#390108]

Support Guide: ZI-540 — NZ Post Enhancements: SOv2 + Label Upgrades + Scoping

From SL: ZI-540 — NZ Post Enhancements: SOv2 + Label upgrades + Scoping (consolidates ZI-541, ZI-543) [#390108]

Trello card: https://trello.com/c/l1oRHe2L/4460

Feature Summary

This card extends the existing NZ Post adapter in MCSL with three additive capability groups, all gated behind a per-account feature toggle. First, **Shipping Options v2 (SOv2) consignment-level pricing** replaces per-parcel client-side summing with a single consignment-level rate call to NZ Post, so multi-parcel orders return a single accurate price from the carrier. Second, two new services are surfaced: **Fliway oversize** (for parcels exceeding NZ Post's oversize threshold) and **Express** (for eligible domestic orders). Third, **Parcel Label v3** enhancements add a platform logo field and a merchant 1D barcode field to returns labels. The toggle-off path preserves all pre-existing NZ Post behaviour without regression. Rollout is pilot-gated to 5–10 NZ Post credit-account merchants before

broad release. This card consolidates ZI-541 and ZI-543; ZI-542 (MyNZPB SMB) is out of scope.

Toggles & Prerequisites

Toggle / config note	What support should confirm
Feature toggle (on): `{accountUUID}.nz.post.v3.domestic.rates.enabled`	Confirm the toggle is enabled for the specific account UUID before testing or troubleshooting new behaviour. Replace `{accountUUID}` with the merchant's actual account UUID.
Feature toggle (off): Same key, value = off	Confirm toggle is off for non-pilot accounts. Legacy SOv2/v2 behaviour must be preserved — no consignment pricing, no Parcel Label v3 fields, no Fliway/Express new services forced in.
Pilot gating	This feature is restricted to a pilot cohort of 5–10 NZ Post credit-account merchants. Confirm the merchant is in the pilot cohort before enabling the toggle. Non-pilot accounts must not see the new flow.
Carrier account prerequisite	NZ Post **credit-account** only. Confirm the merchant has a valid NZ Post credit account configured in hamburger menu > Carriers.
Platform prerequisite	Shopify MCSL is the primary scoped platform. Testing store confirmed: `mcsl-loggedin354.myshopify.com`. Multi-platform support (WooCommerce, BigCommerce, Magento, PrestaShop) should be validated on the reported platform.
Parcel Label v3 — platform logo	Confirm a platform logo is configured in carrier > other details settings before testing TC-4.
Parcel Label v3 — merchant 1D barcode (returns)	Confirm the merchant has configured a returns 1D barcode value in carrier > other details settings before testing TC-5. If the barcode value is missing or malformed, the label must still print (without the barcode).
CPSR (Signature Required) add-on	Confirm the existing CPSR add-on builder configuration is in place. This must continue to apply without regression regardless of toggle state.
Release	MCSL 381 (SL iteration backlog). Confirm the store is on a build that includes MCSL 381.
Consolidated cards	ZI-541 and ZI-543 are consolidated into this card. ZI-542 (MyNZPB SMB) is out of scope — do not conflate.

Step-by-Step Support Walkthrough

The primary test and support platform for this card is **Shopify**. If a ticket is raised on WooCommerce, BigCommerce, Magento, or PrestaShop, support is validating the same behaviour on the reported platform using the equivalent admin path noted in each step.

---

1. Confirm toggle state for the merchant account

› Navigate to the account's toggle/feature-flag configuration (internal admin or support tooling).

› Confirm `{accountUUID}.nz.post.v3.domestic.rates.enabled` is set to **on** for pilot accounts, **off** for all others.

› If the toggle state does not match the expected behaviour, stop here and correct it before proceeding.

---

2. Confirm NZ Post credit-account carrier configuration

› In the MCSL app: **hamburger menu > Carriers**.

› Confirm NZ Post is listed and the credit-account credentials are saved and active.

› On **Shopify**: Shopify Admin > Apps > PH Multi Carrier Shipping Label App > hamburger menu > Carriers.

› On **WooCommerce**: WordPress Admin > PluginHive MCSL app/settings > Carriers.

› On **BigCommerce**: BigCommerce admin > MCSL app/settings > Carriers.

› On **Magento**: Magento admin > PluginHive Multi Carrier Shipping Label extension/settings > Carriers.

› On **PrestaShop**: PrestaShop admin > PluginHive Multi Carrier Shipping Label module/settings > Carriers.

---

3. Verify consignment-level pricing for a multi-parcel domestic NZ order (TC-1)

› Go to the **ORDERS tab**, open a domestic NZ order that contains **3 or more parcels**.

› Click **Get Rates** (Prepare Shipment panel).

› Observe the rates returned in the UI — the displayed rate should reflect a single consignment-level price, not a per-parcel sum.

› Navigate to **hamburger menu > Settings > Request Log** (or the equivalent log viewer for the platform).

› Locate the outbound NZ Post rate request for this order.

› `- Request nodes to verify: confirm a single SOv2 consignment-level rate call is present in the log (not multiple per-parcel calls); confirm the consignment price field in the response matches what is displayed in the UI.`

---

4. Verify Fliway oversize service for an oversize parcel (TC-2)

› Go to the **ORDERS tab**, open or create an order where at least one parcel exceeds the NZ Post oversize threshold.

› Click **Get Rates**.

› Confirm **Fliway oversize** appears as a service option in the rate list and is annotated as an oversize service.

› Select the Fliway service and click **Generate Label**.

› Confirm the label is downloadable.

› Check the request log (**hamburger menu > Settings > Request Log**).

› `- Request nodes to verify: confirm the Fliway oversize service code is present in the rate response; confirm the label request uses the Fliway routing/service code.`

› Also confirm: for a **non-oversize** order, Fliway oversize does **not** appear in the rate list (TC-9).

---

5. Verify Express service code for an eligible domestic order (TC-3)

› Go to the **ORDERS tab**, open a domestic NZ order eligible for Express.

› Click **Get Rates**.

› Confirm an **Express** service appears in the rate list.

› Select Express and click **Generate Label**.

› Check the request log.

› `- Request nodes to verify: confirm the Express service code is present in the rate response; confirm the Parcel Label v3 request body uses the Express service code.`

---

6. Verify Parcel Label v3 — platform logo on generated label (TC-4)

› Confirm a platform logo is configured in **carrier > other details** settings (**hamburger menu > Settings > carrier > other details** or equivalent).

› Go to the **ORDERS tab**, open a domestic NZ order, and click **Generate Label**.

› Download the label PDF/PNG.

› Visually confirm the PluginHive/platform logo is visible on the label.

› Check the request log.

› `- Request nodes to verify: confirm the Parcel Label v3 request body includes the platform-logo field with a non-empty value.`

---

7. Verify Parcel Label v3 — merchant 1D barcode on returns label (TC-5)

› Confirm the merchant has configured a returns 1D barcode value in **carrier > other details** settings.

› Go to the **ORDERS tab**, open an order, and generate a **returns label**.

› Download the returns label artifact.

› Visually confirm the 1D barcode appears in the documented position on the label.

› Check the request log.

› `- Request nodes to verify: confirm the Parcel Label v3 request body includes the merchant 1D barcode field with the configured value.`

› **Edge case:** If the barcode value is missing or malformed, confirm the label still generates and downloads (without the barcode) — no hard failure (TC-5 graceful fallback).

---

8. Verify CPSR (Signature Required) add-on regression (TC-6)

› Go to the **ORDERS tab**, open an order that requires CPSR/Signature Required.

› Confirm the CPSR add-on is applied via the existing add-on builder (**hamburger menu > Carriers** or the add-on builder within the Prepare Shipment panel — confirm path from live store context).

› Click **Get Rates**, then **Generate Label**.

› Confirm the label is generated with signature-on-delivery.

› Check the request log.

› `- Request nodes to verify: confirm the CPSR field/add-on is present in the rate request body regardless of toggle state.`

› Also verify the **special services** from the More Actions panel are accessible and map to the correct service IDs:

Special service	Service ID
Signature Required	10756
Rural Delivery	10757
Overnight Evening Delivery	10758
Same Day Evening Delivery	10759
Saturday Delivery	10760

ZI-531 - From SL: ZI-531 — TForce carrier integration in MCSL (consolidates #270058, #263693, #274726) [#389467]

Support Guide: ZI-531 — TForce Carrier Integration in MCSL

From SL: ZI-531 — TForce carrier integration in MCSL (consolidates #270058, #263693, #274726) [#389467]

Trello card: https://trello.com/c/zZlGsCnW/4461

Feature Summary

This card introduces TForce Freight as a new carrier integration within the PluginHive Multi Carrier Shipping Label (MCSL) app across Shopify, WooCommerce, BigCommerce, Magento, and PrestaShop. The integration supports LTL freight label generation (including Bill of Lading), a full range of freight classes and packing types, pickup and delivery accessorial services, multiple shipment payer options, and a wide variety of document image types and label formats. This is a net-new carrier addition — merchants who ship LTL freight via TForce can now configure, rate, and generate labels directly from MCSL without a third-party workaround. Two known QA failures were recorded at time of testing (Residential Delivery label reflection and Thermal Label ZPL 4×6 format) and should be tracked for resolution status before confirming full feature availability to merchants.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggles detected	Confirm TForce carrier option is visible in hamburger menu > Carriers after release deployment

Toggle / config note	What support should confirm
TForce carrier account credentials required	Merchant must have an active TForce Freight account; confirm account number and credentials are entered in hamburger menu > Carriers
LTL freight-specific fields (Freight Class, Packing Type)	Confirm these fields are available in the Prepare Shipment panel for TForce orders
BOL Print Format (TFF / VICS)	Confirm both formats are selectable in the label generation UI
Document Image Type selection	Confirm all supported label formats are listed; note ZPL (4×6) is not supported for Label document type — confirm this is either hidden or returns a clear error
Shipment Payer options (Prepaid, Collect, Third Party)	Confirm all three options are available and that Third Party billing fields appear when selected
Instruction character limits (Handling: 300, Pickup: 200, Delivery: 300)	Confirm UI enforces these limits and does not allow submission beyond the character cap
Release: SL MCSL 381 Iteration backlog	Confirm this release is deployed on the merchant's store before troubleshooting TForce availability
Test store: mcsl-loggedin354.myshopify.com	Use this store for internal reproduction; do not share credentials externally
Platforms: Shopify, WooCommerce, BigCommerce, Magento, PrestaShop	Feature is cross-platform; confirm the merchant's platform and validate navigation accordingly

---

Step-by-Step Support Walkthrough

Part 1 — Carrier Setup

1. **Navigate to hamburger menu > Carriers** in the MCSL app (path applies across all supported platforms — Shopify Admin > Apps > PH Multi Carrier Shipping Label App; WooCommerce admin > PluginHive MCSL settings; BigCommerce MCSL app settings; Magento admin > PluginHive Multi Carrier Shipping Label extension; PrestaShop admin > PluginHive Multi Carrier Shipping Label module).
2. Confirm **TForce** appears as an available carrier option.
3. Confirm the merchant has entered valid TForce Freight account credentials (account number, username, password). If credentials are missing or incorrect, label generation and rate fetch will fail at the API level.
4. Save the carrier configuration and confirm no error is returned on save.

Part 2 — Instruction Fields

5. Open a test order in the **ORDERS tab > open order > Prepare Shipment**.
6. Locate the **Handling Instruction**, **Pickup Instruction**, and **Delivery Instruction** fields.
7. Confirm the following character limits are enforced in the UI:

› Handling Instruction: **300 characters**

› Pickup Instruction: **200 characters**

› Delivery Instruction: **300 characters**

8. Attempt to enter text beyond each limit and confirm the UI prevents submission or truncates input.

› *Reference QA: Order #10781 — all three instruction limits passed.*

› `- Request nodes to verify: handling instruction, pickup instruction, and delivery instruction fields in the TForce API request payload — confirm character values do not exceed the respective limits.`

Part 3 — Pickup & Delivery Accessorial Services

9. In **ORDERS tab > open order > Prepare Shipment**, locate the pickup and delivery accessorial service options for TForce.

10. Confirm the following **pickup accessorials** are available and selectable (QA reference: Order #10782 — all pickup accessorials passed).

11. Confirm the following **delivery accessorials** are available and test each:

Accessorial	Service Code	QA Order	QA Status
Residential Delivery	RESD	#10784	□ Known Fail — passes in request, not reflected on label
Inside Delivery	INDE	#10785	□ Pass
Liftgate Delivery	LIFD	#10786	□ Pass
Limited Access Delivery	LADL	#10787	□ Pass
Notification Before Delivery	NTFN	#10788	□ Pass
Tradeshow Delivery	TRDS	#10789	□ Pass
Grocery Warehouse Delivery	GWHD	#10790	□ Known Fail — passes in request, not reflected on label

12. For any accessorial the merchant reports as missing from the label, check the request log first before escalating.

› `- Request nodes to verify: accessorial service codes (e.g., RESD, INDE, LIFD, LADL, NTFN, TRDS, GWHD) in the TForce API request payload — confirm the selected service code is present in the outbound request.`

› `- Request/response nodes to verify: confirm whether the TForce API response acknowledges the accessorial; if RESD or GWHD is selected, note this is a known label-reflection issue — escalate with request and response logs attached.`

Part 4 — Freight Class & Packing Type

13. In **ORDERS tab > open order > Prepare Shipment**, locate the **Freight Class** and **Packing Type** fields for TForce.

14. Confirm the following freight classes are available and selectable: 50, 55, 60, 65, 70, 77.5, 85, 92.5, 100, 110, 125, 150, 175, 200, 250, 300, 400, 500.

15. Confirm the following packing types are available: Skid, Box, Crate, Drum, Bundle, Barrel, Bag, Roll, Pallet, Carton, Case, Loose, Other.

16. Generate a label with a selected freight class and packing type combination and confirm both values appear correctly on the generated label.

› `- Request nodes to verify: freight class value and packing type in the TForce API request — confirm the selected class (e.g., "100") and packing type (e.g., "Pallet") are passed correctly.`

› *QA reference: All 18 freight class combinations passed (Orders #10791–#10810, #1800).*

Part 5 — BOL Print Format

17. In **ORDERS tab > open order > Prepare Shipment**, locate the **BOL Print Format** option.

18. Confirm both formats are available:

› **TFF** (TForce Format)

› **VICS** (Industry Standard)

19. Generate a Bill of Lading label using each format and confirm the label is produced without error.

› *QA reference: TFF — Order #10788 □ Pass; VICS — Order #10856 □ Pass.*

Part 6 — Shipment Payer

20. In **ORDERS tab > open order > Prepare Shipment**, locate the **Shipment Payer** field.

21. Confirm all three options are available and selectable:

› **Prepaid** (Shipper Pays)

› **Collect**

› **Third Party**

22. For **Third Party**, confirm that additional billing fields (third-party account number, address) appear in the UI when this option is selected.

23. Generate a label for each payer type and confirm the billing method is reflected correctly on the label.

› `- Request nodes to verify: shipment payer / billing type field in the TForce API request — confirm "Prepaid", "Collect", or "Third Party" is passed correctly with associated account details where applicable.`

› *QA reference: Prepaid — Order #10788 □; Collect — Order #10811 □; Third Party — Order #10812 □.*

Part 7 — Document Image Type (Label)

24. In **ORDERS tab > open order > Prepare Shipment**, locate the **Document Image Type** selector when **Label** is selected as the label type.

25. Confirm the following formats are available and generate successfully: Address Labels with PRO Nos, Address Label w/o PRO Nos, PRO Stickers, Address Label (1×1), and Address Label (2×1).

ZI-552 - From SL: ZI-552 — Block tracking-notification emails when merchant SMTP is not configured [#390467]

Support Guide: ZI-552 — Block Tracking-Notification Emails When Merchant SMTP Is Not Configured

From SL: ZI-552 — Block tracking-notification emails when merchant SMTP is not configured [#390467]

Trello card: https://trello.com/c/nIGCc0JC/4462

Feature Summary

Prior to this change, when a merchant had not configured their own SMTP in MCSL, the app silently fell back to PluginHive's SMTP transport and sent customer-facing tracking-notification emails using a PluginHive sender address. This meant end-customers received shipping updates from an unknown address, and any replies or complaints landed silently in PluginHive's inbox rather than the merchant's. This fix suppresses those customer-facing tracking emails entirely when the merchant's SMTP is not configured (`isSmtpEnable = false`), writes a structured audit log entry per suppressed event, and surfaces a persistent warning banner in MCSL Settings so the merchant knows emails are being held. A replay mechanism allows previously suppressed notifications to be resent once the merchant enables SMTP. This change is carrier-neutral and applies across all supported platforms: Shopify, WooCommerce, BigCommerce, Magento, and PrestaShop.

Toggles & Prerequisites

Toggle / config note	What support should confirm
`.tracking.email.suppress.no.smtp.enabled: true`	Primary feature flag. Confirms suppression is active for this specific merchant account. Verify the exact accountUUID is present in the flag key.
`tracking.email.suppress.no.smtp.enabled.after.store.createdAt.greater.than:`	Date-gated rollout flag. Suppression only activates for stores created after this date unless the per-account flag overrides it. Confirm the store's `createdAt` value relative to this date.
`.tracking.email.suppress.no.smtp.disabled: true`	Per-account kill switch. If this is `true`, suppression is explicitly disabled for this merchant regardless of the global flag. Check this before assuming suppression should be active.
Feature flag mode: `log-only`	When the flag is in log-only mode, the suppression log entry is written but the email is **still sent** via PluginHive fallback (TC-9 legacy behaviour). Confirm which mode is active before troubleshooting missing emails.
Merchant SMTP configuration (`isSmtpEnable`)	Confirm whether the merchant has SMTP configured and enabled in MCSL Settings. Suppression only triggers when `isSmtpEnable = false` or when no `mailSettings` document exists for the account.
Missing `mailSettings` document (legacy merchants)	A merchant with no `mailSettings` record is treated as SMTP not configured. Suppression applies. Confirm from account/store context.
MCSL release prerequisite	This feature is part of MCSL 381 (SL iteration backlog). Confirm the store is running MCSL 381 or later before troubleshooting.
Platform	Applies to Shopify, WooCommerce, BigCommerce, Magento, and PrestaShop. Confirm the merchant's platform when reviewing logs or replicating.

Step-by-Step Support Walkthrough

1. **Confirm the merchant's SMTP configuration status.**

› Navigate to the MCSL app on the merchant's platform:

› **Shopify:** Shopify Admin > Apps > PH Multi Carrier Shipping Label App

› **WooCommerce:** WordPress Admin > PluginHive MCSL app/settings

› **BigCommerce:** BigCommerce admin > PluginHive MCSL app/settings

› **Magento:** Magento admin > PluginHive Multi Carrier Shipping Label extension/settings

› **PrestaShop:** PrestaShop admin > PluginHive Multi Carrier Shipping Label module/settings

› Go to **hamburger menu > Settings** and locate the email/SMTP configuration section.

› Confirm whether SMTP is enabled (`isSmtpEnable = true`) or disabled/absent (`isSmtpEnable = false` or no `mailSettings` record).

2. **Confirm the active feature flag state for this merchant account.**

› Check whether `.tracking.email.suppress.no.smtp.enabled` is set to `true` for this merchant.

› Check whether `.tracking.email.suppress.no.smtp.disabled` is set to `true` — if so, suppression is explicitly off for this account regardless of other flags.

› Check the global date gate: `tracking.email.suppress.no.smtp.enabled.after.store.createdAt.greater.than`. Confirm the store's creation date relative to this value.

› Confirm whether the flag is in **log-only** mode or full suppression mode, as this changes expected behaviour (see TC-9).

› `- Request/log fields to verify: accountUUID, flag key exact match, flag value (true/false), flag mode (log-only vs. enforced), store createdAt vs. date gate value`

3. **Reproduce or verify the suppression behaviour via Re-track.**

› In the MCSL app, go to the **ORDERS** tab.

› Open the relevant order and navigate to **Prepare Shipment / Generate Label** or the active shipment row in the orders grid.

› Click **Re-track** on an active shipment for a merchant with SMTP disabled.

› Confirm that no tracking-notification email is delivered to the end-customer.

› Confirm that the tracking status in the MCSL order detail **does update** to the new carrier status (suppression must not block the status update — TC-4).

4. **Verify the suppression log entry.**

› Navigate to **hamburger menu > Settings / Shipping Rates / Automation or Request Log** (or the equivalent request/audit log view on the merchant's platform).

› Search for log entries associated with the order or shipment in question.

› Confirm the presence of the structured suppression log entry.

› `- Request/log fields to verify: log message exactly "[ZI-552] trackingEmail suppressed: SMTP not configured", accountUUID, subOrderUUID, timestamp, audit collection entry (suppression event persisted)`

5. Verify the regression case — email sends when SMTP is configured (TC-2).

› For a merchant with SMTP configured and enabled, repeat the Re-track step above.

› Confirm the tracking-notification email is delivered to the end-customer via the merchant's SMTP.

› Confirm **no** `[ZI-552]` suppression log entry appears for this event.

› `- Request/log fields to verify: absence of "[ZI-552]" log entry, outbound SMTP transport = merchant's configured SMTP (not PluginHive transport)`

6. Verify cron-based suppression (TC-3).

› For a merchant with SMTP disabled and shipments due for scheduled tracking, confirm that the 6-hourly tracking cron job does not send tracking-notification emails.

› Check the audit log for one `[ZI-552]` suppression entry per affected shipment.

› `- Request/log fields to verify: "[ZI-552] trackingEmail suppressed: SMTP not configured" per shipment, accountUUID, subOrderUUID, cron trigger context`

7. Verify the persistent warning banner in MCSL Settings (TC-7).

› With suppression events recorded for the merchant, navigate to **hamburger menu > Settings** in the MCSL app on the merchant's platform.

› Confirm a persistent warning banner is visible indicating that tracking emails are being suppressed.

› Confirm the banner instructs the merchant to configure SMTP.

› Confirm the **"Replay suppressed tracking emails"** affordance is present on the banner or Settings page.

8. Verify the replay mechanism after SMTP is enabled (TC-8).

› With suppressed events in the audit collection, have the merchant enable SMTP in **hamburger menu > Settings**.

› Click **"Replay suppressed tracking emails"**.

› Confirm that previously suppressed notifications are sent via the merchant's newly configured SMTP.

› Confirm audit entries for those events are marked as replayed.

› Confirm no duplicate audit rows are created for the same suppressed event.

› `- Request/log fields to verify: audit entry status updated to "replayed", outbound SMTP transport = merchant SMTP, no duplicate audit rows for same subOrderUUID`

9. Verify internal/merchant-self emails are not suppressed (TC-6).

› Confirm that internal or merchant-self emails (e.g. error notifications sent to the merchant) continue to be dispatched via PluginHive's SMTP as before.

› Confirm no `[ZI-552]` suppression log entry is recorded for these internal events.

› `- Request/log fields to verify: absence of "[ZI-552]" entry for internal mail events, outbound transport = PluginHive SMTP for internal mail`

10. Verify defense-in-depth — no PluginHive transport leak for customer-facing emails (TC-10).

› When strict suppression mode is active and the tracking helper returns the suppression sentinel, confirm that `sendTrackingNotificationEmail()` does not open an outbound SMTP connection using PluginHive transport to deliver a customer-facing email.

› Confirm the suppression audit entry is written.

› `- Request/log fields to verify: no outbound SMTP connection to PluginHive transport for customer email, suppression audit entry present, suppression sentinel returned by helper`

Expected Behaviour

What support should observe	How to confirm
No tracking-notification email delivered to end-customer when merchant SMTP is disabled	Re-track an active shipment for a merchant with `isSmtpEnable = false`; confirm no email arrives at the customer address
Structured suppression log entry written per suppressed event	Check request/audit log for `[

ZI-530 - From SL: ZI-530 — Welcome-email overhaul for MCSL (sender/reply-to/personalization) [#387845]

Support Guide: ZI-530 — Welcome-Email Overhaul for MCSL

From SL: ZI-530 — Welcome-email overhaul for MCSL (sender/reply-to/personalization) [#387845]

Trello card: https://trello.com/c/m9W9Cblq/4464

Feature Summary

This change overhauled the install and uninstall email flow for MCSL across all supported platforms. Previously, every app install and uninstall automatically created a Zendesk ticket, generating inbox noise for the support team. With this update, install and uninstall events now send a personalised welcome or exit email directly to the merchant from `PluginHive Onboarding` with a `Reply-To` of `support@pluginhive.com`. No Zendesk ticket is created automatically on install or uninstall. A Zendesk ticket is only created if the merchant actively replies to the email, which is then routed through Zendesk's inbound-email channel. Other Zendesk ticket types (SIGNUP, SUBSCRIPTION, PAYMENT_ERROR) are unaffected. This change affects the onboarding/offboarding experience for merchants on Shopify, WooCommerce, BigCommerce, Magento, and PrestaShop.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggles detected	This change is always-on post-deployment; no toggle needs to be enabled
MCSL 381 release deployed	Confirm the store is on MCSL 381 or later before troubleshooting email behaviour
Zendesk inbound-email channel active	Confirm the inbound-email handshake is live in Zendesk so that merchant replies create tickets correctly
Mailer service (`storepepMailer`) reachable	If welcome/exit email is not received, confirm the mailer is not down; a mailer failure should not crash the install route

Toggle / config note	What support should confirm
Merchant email address populated at install	Welcome email personalisation (first name, store URL) is sourced from the AppInstalled payload; confirm the merchant's account has these fields populated
BigCommerce, Magento, PrestaShop install/uninstall flow	QA confirmed testing on Shopify and WooCommerce only; BigCommerce, Magento, and PrestaShop install/uninstall email flow requires live-store verification
Carriers: FedEx, Australia Post	No carrier-specific gating detected for this email change; carrier context is inherited from the broader MCSL 381 release

---

Step-by-Step Support Walkthrough

Shopify and WooCommerce (QA-verified platforms)

1. **Confirm the merchant received a welcome email on install.**

Ask the merchant to check their inbox (and spam folder) for an email from `PluginHive Onboarding` sent at the time of MCSL installation.

Request/log fields to verify: `From: PluginHive Onboarding` , `Reply-To: support@pluginhive.com`, **merchant first name rendered in body, store URL rendered in body, no unreplaced placeholder tokens such as** `{{firstName}}` **visible in the email body.**

2. **Confirm no Zendesk ticket was auto-created at install.**

In Zendesk, search for tickets created around the time of the merchant's install. Filter by the merchant's store URL or email. There should be **no** ticket with a source of the install path (`wss_installation` type or equivalent). If a ticket exists from this path, this is a regression — escalate immediately.

3. **Confirm the welcome email headers pass deliverability checks (Shopify / WooCommerce).**

Ask the merchant to forward the raw email headers or use a tool such as MXToolbox. Verify:

Request/log fields to verify: SPF = `pass`**, DKIM =** `pass`**, DMARC =** `pass`**. If any of these fail, the email may land in spam — escalate to the development team with the raw headers.**

4. **Confirm email personalisation is correct.**

Review the email body the merchant received:

› Merchant's first name should appear in the documented position.

› Store URL should appear in the documented position.

› No placeholder tokens (e.g. `{{firstName}}`) should be visible.

If placeholders are visible, this indicates the AppInstalled payload did not supply the expected fields — collect the store URL and account UUID and escalate.

5. **Confirm the uninstall exit email is sent.**

For Shopify: ask the merchant (or reproduce on a dev store) to uninstall MCSL from **Shopify Admin > Apps > PH Multi Carrier Shipping Label App** and check the merchant inbox for an exit email from the same `PluginHive Onboarding` identity.

For WooCommerce: deactivate/uninstall the plugin from **WordPress Admin / WooCommerce store admin > Plugins** and check the merchant inbox.

Request/log fields to verify: `From: PluginHive Onboarding ,Reply-To: support@pluginhive.com` **present in exit email headers.**

6. **Confirm no Zendesk ticket was auto-created at uninstall.**

Same as step 2 — search Zendesk for tickets created at the time of uninstall. No ticket should exist from the uninstall path.

7. **Confirm that replying to the welcome email creates a Zendesk ticket.**

Ask the merchant (or test on a dev store) to reply to the welcome email. A new Zendesk ticket should be created via the inbound-email channel with:

- The merchant as the requester.
- The reply text in the ticket body.

If no ticket is created on reply, confirm the Zendesk inbound-email channel is active and the `Reply-To: support@pluginhive.com` address is correctly configured in Zendesk. Escalate if the inbound-email handshake is not live.

8. **Confirm reinstall sends exactly one welcome email (no duplicates).**

If a merchant reports receiving duplicate welcome emails after reinstalling MCSL, collect the timestamps of each email received and escalate with the store URL and account UUID.

Request/log fields to verify: Single `AppInstalled` **event in the events stream; no duplicate mailer calls.**

9. **Confirm other Zendesk ticket types are unaffected (regression check).**

Verify that SIGNUP, SUBSCRIPTION, and PAYMENT_ERROR flows still create Zendesk tickets as expected. These paths were explicitly preserved and should not be affected by this change. If a merchant reports missing Zendesk tickets for these types, escalate immediately.

BigCommerce, Magento, and PrestaShop (not QA-verified — live-store confirmation required)

10. **For BigCommerce:** The install/uninstall email flow on BigCommerce has not been QA-verified. If a BigCommerce merchant reports not receiving a welcome or exit email, or reports an unexpected Zendesk ticket being created, treat this as a potential platform-specific gap. Collect the store URL, account UUID, and any available install/uninstall timestamps and escalate to the development team.

Navigate to **BigCommerce admin > MCSL app settings** to confirm the app version and installation state.

11. **For Magento:** Same as BigCommerce — the install/uninstall email flow has not been QA-verified on Magento. Collect store URL, account UUID, and escalate. Navigate to **Magento admin > PluginHive Multi Carrier Shipping Label extension/settings** to confirm the extension version and installation state.

12. **For PrestaShop:** Same as BigCommerce and Magento — not QA-verified. Collect store URL, account UUID, and escalate. Navigate to **PrestaShop admin > PluginHive Multi Carrier Shipping Label module/settings** to confirm the module version and installation state.

Expected Behaviour

What support should observe	How to confirm
Merchant receives one welcome email on install from `PluginHive Onboarding`	Check merchant inbox; verify `From` header matches exactly
`Reply-To` header is `support@pluginhive.com`	Inspect raw email headers forwarded by merchant
Email body contains merchant first name and store URL with no unreplaced placeholder tokens	Review email body text; confirm `{{firstName}}` or similar tokens are absent
SPF, DKIM, and DMARC all show `pass` in email headers	Merchant forwards raw headers or uses MXToolbox; all three authentication results = `pass`
No Zendesk ticket is auto-created on install	Search Zendesk for tickets at install time; none should exist from the install path
Merchant receives one exit email on uninstall from the same PluginHive Onboarding identity	Check merchant inbox after uninstall; verify `From` and `Reply-To` headers
No Zendesk ticket is auto-created on uninstall	Search Zendesk for tickets at uninstall time; none should exist from the uninstall path
Merchant reply to welcome email creates a Zendesk ticket via inbound-email	Reply to welcome email; confirm ticket appears in Zendesk with merchant as requester and reply text in body
Reinstall sends exactly one welcome email, no duplicates	Check merchant inbox after reinstall; only one email present
`AppInstalled` and `AppUnInstalled` events still fire; Slack/dashboard signal produced	Confirm with ops/BI team that install volume signals are unaffected
SIGNUP, SUBSCRIPTION, PAYMENT_ERROR Zendesk ticket types still created as before	Trigger or observe one of these flows; confirm Zendesk ticket is created normally
Mailer failure does not crash the install route (returns non-500)	If mailer is down, install completes; error

ZI-524 - From SL: ZI-524 — Show Amazon Shipping estimated delivery days in order grid [#387563]

Support Guide: ZI-524 — Amazon Shipping Transit Days Column in Order Grid

From SL: ZI-524 — Show Amazon Shipping estimated delivery days in order grid [#387563]

Trello card: https://trello.com/c/Cr6MYVVL/4465

Feature Summary

This change surfaces the transit days returned by Amazon Shipping at rate-shop time as a dedicated, sortable, and filterable **"Amazon Transit days"** column on the MCSL order grid (and in CSV exports). The value is written to the order document as `etaByCarrier.amazon`, sourced from

`promisedDeliveryDate` in the Amazon Shipping rate response. The column is **off by default for existing merchants** and is gated behind the `account_uuid.eta.column.enabled` toggle. This is an Amazon Shipping-specific feature.

Toggles & Prerequisites

Toggle / config note	What support should confirm
`account_uuid.eta.column.enabled: true`	Confirm this toggle is enabled for the merchant's account UUID before troubleshooting any visibility issue. If false, the column will not appear — this is expected behaviour.
Off-by-default for existing merchants	Existing merchants will not see the column after release unless they opt in via order-grid display settings or the toggle is enabled. Confirm the merchant has not assumed it auto-appears.
Amazon Shipping carrier must be configured	Confirm Amazon Shipping is set up and active under hamburger menu > Carriers. Without a configured Amazon Shipping account, no ETA data will be generated or persisted.
Amazon Shipping rate-shop must have occurred post-release	The `etaByCarrier.amazon` field is only written on orders rated after this change ships. Legacy orders will have a blank ETA cell — this is expected. Confirm the order was rated after the release.
MCSL 381 release must be deployed	Confirm the store is running MCSL 381 or later. Confirm from live store/card context if the exact release date is needed.
Platform scope	Feature is implemented across Shopify, WooCommerce, BigCommerce, Magento, and PrestaShop MCSL. Validate on the platform the merchant is reporting from.
`indexConfig.json` — no changes deployed	This card explicitly does not modify the search index config. If a merchant reports broken order search after this release, confirm no other concurrent change touched `indexConfig.json`.

Step-by-Step Support Walkthrough

1. **Confirm the toggle is enabled for the merchant's account.**

Navigate to the internal account configuration and verify `account_uuid.eta.column.enabled: true` is set for the merchant's account UUID. If the toggle is absent or false, the column will not appear and no further steps are needed until it is enabled.

2. **Confirm Amazon Shipping is configured on the merchant's store.**

In the MCSL app — hamburger menu > Carriers — verify that Amazon Shipping is listed and active. If Amazon Shipping is not configured, no ETA data will be returned or persisted.

3. **Open a test or reported order that was rated via Amazon Shipping after the MCSL 381 release.**

In the ORDERS tab, locate an order where Amazon Shipping was used for rate-shopping. Open the order to confirm it was processed post-release (legacy orders will have no ETA value).

4. **Verify the "Amazon Transit days" column appears in the order grid.**

› **Shopify:** Shopify Admin > Apps > PH Multi Carrier Shipping Label App > ORDERS tab. Confirm the "Amazon Transit days" column is visible in the order grid.

› **WooCommerce:** WordPress Admin > PluginHive MCSL app > ORDERS tab. Confirm the column is visible.

› **BigCommerce:** BigCommerce admin > PluginHive MCSL app > ORDERS tab. Confirm the column is visible.

› **Magento:** Magento admin > PluginHive Multi Carrier Shipping Label extension > ORDERS tab. Confirm the column is visible.

› **PrestaShop:** PrestaShop admin > PluginHive Multi Carrier Shipping Label module > ORDERS tab. Confirm the column is visible.

If the column is not visible, ask the merchant to check their order-grid display settings and confirm the "Amazon Transit days" column has been enabled there. Also re-confirm the toggle state from Step 1.

5. **Verify the ETA value displays correctly in the grid.**

› For orders rated via Amazon Shipping post-release: the "Amazon Transit days" cell should show a numeric value in days (e.g., `3`).

› For orders rated via FedEx, UPS, or any non-Amazon carrier: the "Amazon Transit days" cell should be blank.

› For legacy Amazon Shipping orders rated before this release: the cell should be blank or show "—". This is expected.

6. **Verify sort and filter behaviour on the "Amazon Transit days" column.**

› Sort the order grid by "Amazon Transit days" ascending. Orders should list in ascending ETA order (e.g., 1, 3, 5). Orders without an ETA value should group consistently at the top or bottom per the implemented sort policy.

› Apply a filter such as "Amazon Transit days ≤ 3 days". Only orders with an ETA of 3 days or fewer should appear. Orders without an ETA value should be excluded from filtered results. Clearing the filter should restore the full view.

7. **Verify the order summary drawer still shows ETA correctly (regression check).**

Open an Amazon Shipping order from the ORDERS tab and expand the order summary drawer. Confirm the ETA value displayed in the drawer matches the value shown in the "Amazon Transit days" grid column.

8. **Verify toggle-off behaviour.**

Disable the `account_uuid.eta.column.enabled` toggle (or have the merchant disable the column from display settings). Reload the order grid and confirm the "Amazon Transit days" column is no longer visible and the existing column order and widths are preserved.

Expected Behaviour

What support should observe	How to confirm
"Amazon Transit days" column is visible in the order grid when the toggle is enabled	ORDERS tab on the merchant's platform — column appears in the grid header row

What support should observe	How to confirm
"Amazon Transit days" column is **not** visible when the toggle is disabled or the merchant has not opted in	Disable toggle or check display settings — column absent, existing columns unaffected
Amazon Shipping orders rated post-release show a numeric ETA value (days) in the column	Open an Amazon-rated order in the ORDERS tab; cell shows e.g. 3
Non-Amazon carrier orders (FedEx, UPS, etc.) show a blank cell in the "Amazon Transit days" column	View FedEx or UPS orders in the grid — "Amazon Transit days" cell is empty
Legacy orders (rated before this release) show blank or "—" in the "Amazon Transit days" cell	Open a pre-release order — no `etaByCarrier.amazon` value exists; cell renders gracefully with no console error
Order grid sorts correctly by "Amazon Transit days" ascending/descending	Sort the column — orders with values sort numerically; orders without values group consistently
Filter by "Amazon Transit days ≤ N days" returns only matching orders; clearing restores full view	Apply and clear filter in the ORDERS tab
Order summary drawer ETA display is unaffected (regression)	Open order drawer — ETA value matches the grid column value
Order search results are unaffected (regression — no `indexConfig.json` changes)	Run historical order searches — results match pre-release behaviour
Existing column order and widths are preserved for merchants who have not opted in	Open order grid on an account without the toggle — layout unchanged

---

Merchant-Safe Explanation

Amazon Shipping now provides estimated transit days when it rates your orders. That value is saved on each order and shown as a new **"Amazon Transit days"** column in your order list, which you can sort, filter, and include in CSV exports. The column is optional — turn it on or off from your order-grid display settings. Orders placed before this update, or shipped via other carriers, will show a blank cell.

ZI-523 - From SL: ZI-523 — Amazon Shipping label-failure error message mis-mapped [#387563]

Support Guide: ZI-523 — Amazon Shipping Label-Failure Error Message Mis-Mapped

From SL: ZI-523 — Amazon Shipping label-failure error message mis-mapped [#387563]

Trello card: https://trello.com/c/Gvyyzmdl/4466

Feature Summary

Prior to this fix, when a rate-fetch failed for Amazon Shipping (or any other carrier), the order row always displayed the generic message **"No Shipping Service selected. Please check your Automation**

Rules." — even when the failure was caused by a real carrier-side error such as a marketplace mis-configuration, out-of-coverage address, or weight/dimension violation. The actual carrier error was silently swallowed. This fix refactors `buildErrorUpdateObject()` so that when a genuine carrier-side error is present, it is surfaced verbatim on the order row with a carrier name prefix (e.g. `Amazon Shipping: InvalidInput: marketplace not configured`). The generic fallback message is preserved only for the true no-carrier-configured case. The fix is framework-level and applies to all carriers, but the customer-reported regression is Amazon Shipping. Affected flows include single-order rate fetch, bulk rate fetch, and automation-rule-triggered rate fetch across all supported platforms.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggles detected	No gating — fix is active for all stores on MCSL 381+
Release prerequisite	Store must be on MCSL 381 or later; confirm plugin/app version before troubleshooting
Amazon Shipping carrier account	Confirm Amazon Shipping is connected under hamburger menu > Carriers; marketplace must be configured to reproduce the mis-config scenario
Spot-check carriers (FedEx, UPS, DHL Express, USPS, Canada Post, Australia Post)	Each must be connected and eligible for the test order to verify the generic fix applies beyond Amazon Shipping
Automation Rules (if testing TC-7)	Confirm at least one active rule routes orders to Amazon Shipping or the failing carrier; check hamburger menu > Settings / Automation
Bulk rate-fetch (if testing TC-6)	Confirm multiple orders are present in the ORDERS tab and at least two are expected to fail
Customer/test platform	Shopify MCSL (primary); fix is shared at the framework layer — validate on the reported platform (WooCommerce, BigCommerce, Magento, or PrestaShop) if the ticket originates there

---

Step-by-Step Support Walkthrough

Part A — Verify the Fix: Amazon Shipping Mis-Config Surfaces Real Error

1. **Open the MCSL app** on the merchant's platform:

› **Shopify:** Shopify Admin > Apps > PH Multi Carrier Shipping Label App

› **WooCommerce:** WordPress Admin > PluginHive MCSL app/settings

› **BigCommerce:** BigCommerce admin > MCSL app/settings

› **Magento:** Magento admin > PluginHive Multi Carrier Shipping Label extension/settings

› **PrestaShop:** PrestaShop admin > PluginHive Multi Carrier Shipping Label module/settings

2. **Confirm Amazon Shipping is connected but mis-configured** (e.g. marketplace not set up):

› Navigate to **hamburger menu > Carriers**

› Locate Amazon Shipping and confirm the marketplace field is intentionally left unconfigured or set to an invalid value for testing purposes.

3. **Open the ORDERS tab**, locate an order eligible for Amazon Shipping, and click **Get Rates** (or open the order and use **Prepare Shipment / Generate Label**).

4. **Observe the order row / summary message** after the rate-fetch attempt completes.

› **Expected (post-fix):** The order row shows the real Amazon Shipping error, e.g. `Amazon Shipping: InvalidInput: marketplace not configured`.

› **Not expected:** The generic `No Shipping Service selected. Please check your Automation Rules.` message.

5. **Check the request log** to confirm the carrier error was captured:

› Navigate to **hamburger menu > Settings > Request Log** (path may vary by platform).

› Filter by the order number or timestamp.

› `- Request/log fields to verify: carrier error string (e.g. "InvalidInput: marketplace not configured"), carrier name prefix ("Amazon Shipping:"), order ID, and timestamp of the failed rate-fetch call.`

Part B — Regression Check: No Carrier Configured Still Shows Generic Message

6. **Temporarily remove or disable all carriers** under **hamburger menu > Carriers** (or use a test order with no eligible carrier).

7. **Open the ORDERS tab**, locate an order, and click **Get Rates**.

8. **Observe the order row message.**

› **Expected:** `No Shipping Service selected. Please check your Automation Rules.`

› **Not expected:** Any fabricated carrier string or blank message.

9. **Confirm no carrier API call was made** by checking the request log — no outbound carrier request should appear for this order at this timestamp.

› `- Request/log fields to verify: absence of any carrier API request entry for the order; only the internal no-carrier event should be logged.`

Part C — Regression Check: Timeout / Empty Error Falls Back Gracefully

10. **Simulate or identify a carrier timeout scenario** (e.g. a carrier endpoint that is unreachable or a known timeout condition in staging).

11. **Click Get Rates** on an eligible order.

12. **Observe the order row message.**

› **Expected:** Generic message or a documented `rate fetch timed out` message — not a fabricated carrier string.

› `- Request/log fields to verify: timeout event entry in the request log; confirm no carrier-specific error string is fabricated when the carrier response is empty or absent.`

Part D — Multi-Carrier Scenarios

13. **Configure two carriers** — one expected to succeed (e.g. UPS) and one expected to fail (e.g. Amazon Shipping with mis-config) — under **hamburger menu > Carriers**.

14. **Open the ORDERS tab**, select an order eligible for both, and click **Get Rates**.

15. **Observe the order row:**

› UPS rates should be returned and displayed.

› The Amazon Shipping error should appear in the request log.

› No false generic error should appear on the order row for the UPS-successful result.

› `- Request/log fields to verify: UPS rate response nodes (service name, rate amount); Amazon Shipping error string with carrier prefix; confirm both entries are present in the same request log session.`

16. **For the all-fail scenario (TC-5):** Configure both carriers to fail with distinct errors. After clicking **Get Rates**, confirm the order row shows carrier-prefixed errors for each carrier and the generic fallback is not used.

› `- Request/log fields to verify: distinct carrier error strings per carrier, each with the correct carrier name prefix.`

Part E — Bulk Rate Fetch

17. **In the ORDERS tab**, select 5 orders (3 expected to succeed, 2 expected to fail with carrier errors).

18. **Click Get Rates** for the bulk selection.

19. **Observe each order row individually:**

› The 3 successful orders should display rates.

› The 2 failing orders should display their carrier-specific errors on the row/summary.

› The bulk operation should complete without aborting the successful orders.

› `- Request/log fields to verify: per-order carrier error strings on failing rows; rate data on successful rows; confirm no bulk abort event in the log.`

Part F — Automation Rule Path

20. **Navigate to hamburger menu > Settings / Automation** and confirm an active rule routes orders to Amazon Shipping (or the failing carrier).

21. **Trigger a rate fetch** via the automation rule on an eligible order.

22. **Observe the order row error message** — it should reflect the carrier-side error, not the generic message.

23. **Check the audit trail / request log** to confirm both the rule trigger and the carrier error are recorded together.

› - Request/log fields to verify: automation rule identifier, carrier error string, order ID, and timestamp — all present in the same log entry or adjacent entries.

Part G — Spot-Check Additional Carriers (FedEx, UPS, DHL Express, USPS, Canada Post, Australia Post)

24. **For each carrier listed**, configure it with an intentional mis-configuration under **hamburger menu > Carriers**.

25. **Open the ORDERS tab**, select an eligible order, and click **Get Rates**.

26. **Confirm the order row shows that carrier's specific error with the carrier name prefix** (e.g. `FedEx: ...`, `UPS: ...`, `DHL: ...`).

27. **Confirm the generic fallback is not used** when a real carrier error is present.

› - Request/log fields to verify: carrier-specific error string with correct carrier name prefix for each carrier tested.

Part H — Quick Ship and Label-Fulfill-Order Paths

28. **Use the Quick Ship flow** on an order where the carrier is configured to fail. Confirm the carrier-side error surfaces on the order — not the generic message.

29. **Use the label-fulfill-order path** on a separate order with the same failing carrier. Confirm the same carrier-side error surfaces.

30. **Check the request log for both flows** to confirm neither path silently swallows the carrier error.

› - Request/log fields to verify: carrier error string present in the log for both the Quick Ship entry and the label-fulfill-order entry; confirm no silent swallow (i.e. no entry where the error is absent but the order shows the generic message).

Expected Behaviour

What support should observe	How to confirm
Amazon Shipping mis-config error appears verbatim on the order row with carrier prefix (e.g. `Amazon Shipping: InvalidInput: marketplace not configured`)	ORDERS tab > order row / summary message after Get Rates
Generic `	

ZI-555 - From SL: ZI-555 — MCSL: FedEx REST postalCode null fix for HK / Taiwan (Cybershaft) [#391078]

Support Guide: ZI-555 — FedEx REST postalCode Null Fix for HK / Taiwan (Cybershaft)

Card reference: https://trello.com/c/Hvkv3JgI · Release: SL MCSL 381 Iteration Backlog

---

Feature Summary

This card is part of the MCSL 381 iteration backlog and is tracked under the linked Trello reference https://trello.com/c/Hvkv3JgI. The specific feature change, acceptance criteria, and implementation details have not been populated in the card description, QA notes, checklists, or test cases at the time this guide was generated. Support should treat this as a **pending-detail card** and confirm the exact scope directly from the card owner or release notes before attempting merchant-facing verification. The affected platform is Shopify, and no specific carrier has been identified as in scope.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggles detected	Confirm from live card context whether any feature flag or config toggle gates this change
Release prerequisite	Confirm the store is on MCSL 381 or later before attempting verification
Carrier account prerequisite	No specific carrier identified — confirm from card owner if carrier-specific setup is required
Customer / test platform	Shopify — verify using Shopify Admin > Apps > PH Multi Carrier Shipping Label App
Card description / AC	Not populated — confirm full acceptance criteria from card owner or release manager before support walkthrough

---

Step-by-Step Support Walkthrough

> □□ **Important:** The card description, acceptance criteria, QA notes, and test cases are not populated. The steps below represent the standard verification baseline for a Shopify-platform card in MCSL 381. Update these steps once card details are confirmed.

1. **Confirm the app version on the merchant's store.**

› Navigate to **Shopify Admin > Apps > PH Multi Carrier Shipping Label App**.

› Confirm the installed version is MCSL 381 or later. If the store is on an earlier version, the change may not be present.

2. **Confirm the card scope with the release or development owner.**

› Pull the full card from https://trello.com/c/Hvkv3JgI.

› Identify the exact feature area: whether it affects the **ORDERS tab**, **hamburger menu > Carriers**, **hamburger menu > Products**, checkout rates, label generation, reports, or another area.

› Do not proceed with merchant-facing verification until the scope is confirmed.

3. **Navigate to the identified feature area inside MCSL on Shopify.**

› Once scope is confirmed, open **Shopify Admin > Apps > PH Multi Carrier Shipping Label App** and go to the relevant section (ORDERS, hamburger menu > Carriers, hamburger menu > Products, or as directed by the card).

› Verify the behaviour described in the card's acceptance criteria is present and functioning as expected.

4. **Cross-check Shopify source data if order or product data is involved.**

› Navigate to **Shopify Admin > Orders** or **Shopify Admin > Products** as appropriate.

› Confirm that data displayed or processed inside MCSL matches the Shopify source record.

5. **Inspect request/response logs if carrier API behaviour is in scope.**

› Inside MCSL, navigate to the **ORDERS tab** and open the relevant order.

› Check the request/response log for the relevant carrier call.

› *(Specific request nodes to verify: Confirm from live card context — not available at guide generation time.)*

6. **Document findings and compare against the card's acceptance criteria.**

› Record the observed behaviour, any screenshots, and the app version.

› If behaviour does not match the AC, capture the full log and escalate per the packet below.

Expected Behaviour

What support should observe	How to confirm
Feature behaves as described in card AC	Confirm from live card context — AC not populated at guide generation time
No regression in adjacent MCSL features (ORDERS, Carriers, Products, label generation)	Spot-check ORDERS tab and hamburger menu > Carriers on the test/merchant store
Shopify order and product data renders correctly inside MCSL	Cross-check Shopify Admin > Orders and Shopify Admin > Products against MCSL display
No unexpected errors or UI breakage on Shopify platform	Navigate through MCSL on Shopify Admin and confirm no console errors or broken UI states
Request/response log nodes (if carrier-specific)	Confirm specific node names from card owner — not available at guide generation time

---

Merchant-Safe Explanation

This update is part of a planned improvement released in MCSL 381. The specific change it delivers will be confirmed once the card details are available — your support contact will follow up with a clear explanation of what has changed and how it may affect your shipping workflow on Shopify. No action is required on your end unless your support engineer advises otherwise.

Common Questions & Troubleshooting

› **Check first:** Confirm the store is running MCSL 381 or later. Many reported issues on backlog cards are version-related.

› **Check second:** Pull the full card from https://trello.com/c/Hvkv3JgI to confirm the AC before telling the merchant what to expect.

› **If the feature is not visible:** Confirm whether a toggle or config flag is required — none was detected at guide generation time, but this should be verified with the development owner.

› **If Shopify order or product data looks wrong inside MCSL:** Cross-check Shopify Admin > Orders or Shopify Admin > Products to isolate whether the issue is in Shopify source data or in MCSL processing.

› **When to inspect logs:** If the card involves carrier API calls or rate/label generation, open the ORDERS tab in MCSL, locate the relevant order, and review the request/response log. Capture the full log payload before escalating.

› **When to escalate:** Escalate immediately if the card AC cannot be confirmed from the Trello card, if the feature is not present on a store confirmed to be on MCSL 381, or if a regression is observed in an adjacent feature area.

› **Do not invent scope:** If the card description remains empty, do not attempt to guess the feature behaviour for the merchant. Escalate to the release owner for clarification first.

Support Escalation Packet

› **Story card:** https://trello.com/c/Hvkv3JgI · Parent/backlog card: https://trello.com/c/3rJJ9BuY/4506 · Release: SL MCSL 381 Iteration Backlog

› **Store URL:** *(Confirm from merchant ticket or live store context)*

› **Account UUID:** *(Confirm from MCSL account record)*

› **Carrier account:** *(Not identified — confirm from card owner if carrier-specific)*

› **Order number:** *(Provide the specific Shopify order number used during verification)*

› **Generated label / tracking / report identifier:** *(Attach if label or report generation is in scope)*

› **Screenshots required:** Setup page state inside MCSL on Shopify Admin > Apps > PH Multi Carrier Shipping Label App; request/response log if carrier behaviour is involved; Shopify Admin > Orders or Products cross-check if data accuracy is in scope

› **Toggle / config value:** None detected — confirm from card owner before escalating if a gate is suspected

› **Card AC status:** Not populated at guide generation time — attach confirmed AC text when escalating so the engineering team can validate against implementation

ZI-520 - From SL: ZI-520 — Amazon Shipping India Verified Address (VA) API integration [#389826]

Support Guide: ZI-520 — Amazon Shipping India Verified Address (VA) API Integration

Card reference: MCSL 381 Iteration Backlog — https://trello.com/c/vdVlxKqX (linked via https://trello.com/c/WSIuurMp/4510)

Feature Summary

This card is part of the MCSL 381 iteration backlog and is scoped to the Shopify platform. The full description, acceptance criteria, and QA evidence were not available at the time this guide was authored — the Trello card body, checklists, test cases, and QA sign-off are all currently empty or unpopulated. Support should treat this guide as a **placeholder framework** that must be completed once the card details are confirmed from the live Trello board or the development/QA team. The carrier scope is not explicitly stated, so this is treated as carrier-neutral until confirmed. No feature toggles have been detected.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggles detected	Confirm with dev/QA whether any feature flag or config switch gates this change
Release: MCSL 381 iteration backlog	Confirm the store is running an MCSL build that includes the 381 iteration deployment
Carrier account prerequisite	Not stated — confirm from live card or dev context whether a specific carrier account is required
Shopify platform confirmed	Validate the merchant's store is on Shopify and the PH Multi Carrier Shipping Label app is installed and active
Card description missing	Retrieve full card details from https://trello.com/c/vdVlxKqX before proceeding with store-level verification

---

Step-by-Step Support Walkthrough

> □□ **Note:** The card description, acceptance criteria, test cases, and QA evidence are all unpopulated. The steps below represent the standard verification framework for a Shopify-scoped MCSL change. Update each step once card details are confirmed.

1. **Confirm the MCSL build version on the merchant's store.**

› Navigate to **Shopify Admin > Apps > PH Multi Carrier Shipping Label App**.

› Check the app version or build identifier visible in the app footer or settings area.

› Confirm the deployed build corresponds to the MCSL 381 iteration release.

2. **Retrieve the full card description and acceptance criteria.**

› Open https://trello.com/c/vdVlxKqX directly.

› Confirm the card title, description, and checklist items are now populated.

› If still empty, escalate to the assigned developer or QA owner before proceeding.

3. **Identify the affected area within the MCSL app on Shopify.**

› Based on the card details (once retrieved), navigate to the relevant section:

› **ORDERS tab** — if the change affects order processing, label generation, or tracking.

› **Hamburger menu > Carriers** — if the change affects carrier configuration or rate logic.

› **Hamburger menu > Products** — if the change affects product-level shipping settings.

› **Settings or Reports** — if the change affects configuration defaults or reporting output.

› Confirm from live card context which navigation path applies.

4. **Cross-check source data in Shopify Admin.**

› Navigate to **Shopify Admin > Orders** or **Shopify Admin > Products** as appropriate.

› Verify that the relevant order, product, or shipping data in Shopify Admin aligns with what the MCSL app is reading or displaying.

› Note any discrepancy between Shopify source data and MCSL app behaviour.

5. **Review request/response logs if carrier or rate behaviour is involved.**

› Within the MCSL app, navigate to the **request logs** section (accessible via the ORDERS tab or the relevant carrier's log view).

› Confirm from the card details which specific request nodes or response fields are expected to change.

› *(Log node names cannot be specified until card description is confirmed.)*

› `- Request/log fields to verify:` Confirm from live card or QA evidence — not determinable from current context.

6. **Reproduce the reported behaviour or verify the fix on a test order.**

› Place or locate a test order on the Shopify store that exercises the scenario described in the card.

› Confirm the before/after behaviour matches the acceptance criteria once retrieved.

› Document the order number and any generated label, tracking number, or report identifier for the escalation packet.

Expected Behaviour

What support should observe	How to confirm
Feature behaves as described in card AC	Retrieve AC from https://trello.com/c/vdVlxKqX and validate against live store
No regression in existing MCSL Shopify functionality	Spot-check label generation, rate display, and order sync on the merchant's store
No carrier-specific errors introduced	Review request logs in the MCSL app for any new error codes or failed requests post-deployment
UI reflects expected change (area TBC)	Confirm from card description which UI element or field is affected
Request/log nodes behave as expected	Confirm exact node names from QA evidence once card is populated

---

Merchant-Safe Explanation

This update is part of a scheduled improvement release for the PH Multi Carrier Shipping Label app on your Shopify store. The specific change addresses an item identified during our development iteration

cycle. Once the full details of this update are confirmed internally, this explanation will be updated to reflect exactly what changed and how it may affect your shipping workflow. If you are experiencing unexpected behaviour following a recent app update, please contact support with your store URL and a relevant order number so we can investigate promptly.

Common Questions & Troubleshooting

› **What to check first:** Confirm the MCSL 381 build is deployed on the merchant's store before investigating any reported behaviour. An outdated build will make the change invisible.

› **Card details are empty:** Do not attempt store-level verification until the Trello card at https://trello.com/c/vdVlxKqX is populated with a description and acceptance criteria. Escalate to the QA team or assigned developer immediately.

› **No toggles detected:** If the change is not visible on a qualifying store, confirm there is no undocumented feature flag by checking with the development team — the toggle detection may be incomplete given the empty card state.

› **When to inspect logs:** If the card (once retrieved) involves carrier requests, rate calculation, or label generation, pull request/response logs from the MCSL app's log viewer and compare node values against the expected AC.

› **Regression concerns:** If a merchant reports broken label generation, missing rates, or order sync issues following the 381 deployment, treat it as a potential regression and escalate with full log evidence — do not assume it is related to this specific card without confirming the AC first.

› **When to escalate:** Escalate immediately if the card description remains empty, if QA sign-off cannot be confirmed, or if store-level behaviour cannot be reconciled with the expected outcome once AC is retrieved.

Support Escalation Packet

› **Story card:** MCSL 381 Iteration Backlog — https://trello.com/c/vdVlxKqX (parent reference: https://trello.com/c/WSIuurMp/4510)

› **Store URL:** Confirm from merchant ticket or account record

› **Account UUID:** Confirm from merchant account in the PluginHive backend

› **Carrier account:** Not applicable until card scope is confirmed — confirm from live card context

› **Order number:** Capture from the test or merchant order used to reproduce/verify the behaviour

› **Generated label / tracking / report identifier:** Capture after label generation or report run on the affected order

› **Screenshots required:** App settings page showing the relevant configuration area (TBC from card); request/response log screenshot if carrier behaviour is involved

› **Toggle/config value:** None detected — confirm with dev team whether any undocumented gate exists

› **QA sign-off status:** Not confirmed — QA evidence and test cases are currently unpopulated; escalate to QA team to obtain sign-off before closing any merchant-reported issue tied to this card

Carrier Integration India Post - Carrier Integration India Post

Support Guide: MCSL 381 — Carrier Integration India Post

Carrier Integration India Post

Trello card: https://trello.com/c/WOth7i6s/4537

Feature Summary

This card introduces India Post (Department of Posts) as a new carrier integration in MCSL, covering carrier registration, rate retrieval, label generation, and shipment tracking. Two service types are supported: **SP_INLAND (Speed Post)** and **BUSINESS_PARCEL**, including bulk-customer account routing. The integration spans four PRs across the storepep-react UI, carrier-registration-api, ship-rate-track proxy, and storepep-tracking-api. The feature is flag-gated and was developed and QA-verified against Shopify MCSL, with cross-carrier regression coverage required to confirm no impact on FedEx, Australia Post, or NZ Post flows.

Toggles & Prerequisites

Toggle / config note	What support should confirm
Feature flag — India Post	Flag must be enabled for India Post to appear in "Add a new carrier." Confirm flag state before any troubleshooting. If flag is off, India Post will not appear in the carrier list and no India Post API calls will be made.
Release prerequisite	Card is scoped to MCSL 381 (SL MCSL 381: Iteration backlog). Confirm the store is on a build that includes this release before investigating India Post issues.
Carrier credentials	Merchant must supply: `username`, `password`, `customer id`. QA-verified credentials: username `9999999999`, password `Dop@1234`, customer id `3000064781`. Confirm from live store/card context for production merchants.
Contract selection — SP_INLAND	Contract `41585456` must be selected/bound at registration for Speed Post service. Confirm the correct contract id appears in the request body.
Contract selection — BUSINESS_PARCEL	Contract `41367422` must be selected/bound at registration for Business Parcel service. Confirm the correct contract id appears in the request body.
Bulk customer id	If the merchant is a bulk-customer account, `bulk customer id` (QA value: `1000000444`) must be configured. Confirm the bulk id — not the standard customer id — appears in rate/ship request bodies for bulk flows.
Customer/test platform	Shopify MCSL (primary verified platform). Cross-platform support (WooCommerce, BigCommerce, Magento, PrestaShop) — confirm from live store/card context.
Pickup after label generation	**Not supported.** Pickup settings are present in Carrier Other Details (for pre-label configuration only) but pickup cannot be initiated after label generation. Do not advise merchants to attempt post-label pickup via India Post.
Return label generation	**Not supported.** Do not advise merchants to attempt return label generation for India Post.
Cross-carrier regression flag	QA AC requires regression coverage for FedEx, Australia Post, and NZ Post. If any of these carriers show unexpected behaviour after MCSL 381 deployment, treat as a regression and escalate.

Step-by-Step Support Walkthrough

Part 1 — Carrier Registration

1. In the MCSL app on the merchant's Shopify store, navigate to **hamburger menu → Carriers**.
2. Click **Add a new carrier** and confirm **India Post** appears in the carrier list.

› If India Post is not listed, confirm the feature flag is enabled before proceeding.

3. Enter the India Post credentials: `username`, `password`, and `customer id`. For bulk-customer accounts, also enter the `bulk customer id`.
4. Select the contract at registration:

› **SP_INLAND** → contract `41585456`

› **BUSINESS_PARCEL** → contract `41367422`

5. Save the connector and confirm India Post appears in the active carrier list with the correct service type and contract bound.

› **Request/log fields to verify:** carrier-registration-api call succeeds; connector saved with correct `contract id` and `service_type`; no half-configured state if credentials are invalid.

Part 2 — Carrier Other Details

6. With the India Post connector open in **hamburger menu → Carriers**, locate the **Carrier Other Details** section.
7. Confirm the following nodes are present and configurable:

› Default Package Dimensions

› Pickup or Drop Off

› Shape of Article

› Non-Delivery Instruction

› Delivery Instruction

› Pickup Slot

› Pickup Address

8. Remind the merchant (and note internally) that **Pickup settings here are for pre-label configuration only** — pickup cannot be triggered after label generation.

Part 3 — Packaging Configuration

9. Navigate to **hamburger menu → Products** (or the relevant packaging/product settings area) and confirm the merchant's packaging type is configured.
10. QA has verified all five packaging types for India Post:

› Box

› Quantity

› Stack

› Weight Based

› Weight & Volume Based

11. Confirm the correct packaging type is selected for the order being tested. Weight round-off has been verified as working as expected.

Part 4 — Rates at Checkout

12. On the merchant's Shopify storefront or via the MCSL **ORDERS** tab, trigger a rate request for a domestic India order.

13. Confirm India Post rates are returned for the active service type (SP_INLAND or BUSINESS_PARCEL).

14. Navigate to **hamburger menu** → **Settings** → **Request Log** and locate the India Post rate request.

› **Request nodes to verify:** origin, destination, weight, contract id (`41585456` for SP_INLAND or `41367422` for BUSINESS_PARCEL), `customer_id` (standard or bulk as applicable), `service_type`.

15. Confirm no other carrier (FedEx, Australia Post, NZ Post) is called as part of the India Post rate flow.

Part 5 — Label Generation

16. In the MCSL **ORDERS** tab, open the relevant order and click **Prepare Shipment / Generate Label**.

17. Select the India Post rate and generate the label.

18. Confirm:

› The India Post label is produced and downloadable.

› A tracking number is stored on the MCSL order.

› The label includes the required Speed Post barcode (for SP_INLAND orders).

› **Request nodes to verify:** contract id, `service_type`, `customer_id` (or `bulk_customer_id` for bulk accounts), packaging dimensions, weight, Carrier Other Details nodes (shape of article, delivery instruction, non-delivery instruction).

Part 6 — More Actions (POD, Signature, PIN on Delivery)

19. On the generated label/order in the MCSL **ORDERS** tab, locate the **More Actions** options.

20. Confirm the following nodes pass correctly in the label request when selected:

› **Proof of Delivery (POD):** `ack: "TRUE"`

› **Signature on Delivery:** `reg: "TRUE"`

› **PIN on Delivery:** `otp: "TRUE"`

› **Request nodes to verify:** `ack`, `reg`, `otp` values in the label request payload.

Part 7 — COD (Cash on Delivery)

21. For COD orders, confirm the following COD modes are working:

› **Partial COD**

› **COD + Shipping price**

› **Full COD**

22. In the request log, confirm the COD payload nodes are present for COD orders:

› **Request nodes to verify:** `codr_cod: "COD"`, `value_for_codr_cod: "100"` (or the applicable COD amount).

23. For prepaid orders, confirm the prepaid node is passing correctly in the request body.

› **Request nodes to verify:** prepaid indicator node present and correct for prepaid orders; absent or set to non-COD value for prepaid flows.

Part 8 — Insurance

24. Confirm insurance is configurable and flows correctly from two sources:

› **From product settings:** navigate to **hamburger menu** → **Products**, open the relevant product, and confirm the insurance value is set.

› **From general settings** → **Shipping Settings:** navigate to **hamburger menu** → **Settings** → **Shipping Settings** and confirm the insurance value is set at the global level.

25. Verify the insurance value appears correctly in the label request payload.

› **Request nodes to verify:** insurance/declared value node in the label request.

Part 9 — Fulfillment and Manifest

26. After label generation, confirm the order is marked as fulfilled in **Shopify Admin** → **Orders**.

27. Confirm the manifest is generated correctly for India Post shipments. Navigate to the manifest/reports area in MCSL and verify the India Post shipment appears.

Part 10 — Bulk Customer Routing

28. If the merchant has a bulk customer account, confirm the `bulk customer id` is configured in the India Post connector settings (**hamburger menu** → **Carriers** → **India Post connector**).

29. Trigger a rate and label request under the bulk customer context.

30. In the request log, confirm:

› **Request nodes to verify:** `customer_id` field carries `1000000444` (bulk id), not `3000064781` (standard id). No leakage of the standard customer id in bulk flows.

Carrier Integration DHL Express - Carrier Integration DHL Express

Support Guide: DHL Express REST Carrier Integration

Carrier Integration DHL Express — MCSL 381 Iteration Backlog (Card #4538)

Trello card: https://trello.com/c/LsmHAIIj/4538

Feature Summary

This card introduces a new REST-based DHL Express carrier integration (Carrier Code: **C42**) into PluginHive MCSL, replacing the legacy SOAP/XML DHL Express integration for new accounts. The integration covers carrier setup and credential management, rate retrieval, shipping label and return label generation, special services, tracking, pickup scheduling, shipment cancellation, address validation, and manifest across Shopify, WooCommerce, BigCommerce, Magento, and PrestaShop. Several features confirmed in the legacy SOAP integration are now validated in REST, with a small number of known gaps (COD not supported, Service Point not implemented, Saturday Delivery label generation failing due to account limitations). Support should treat this as the current active DHL Express integration path for new account connections.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggles detected	Confirm from live store/card context whether any gating exists at account or plan level
Carrier Code: **C42**	Confirm the merchant's DHL Express carrier is registered under C42 (REST), not the legacy SOAP carrier code
DHL Express REST replaces SOAP for new accounts	Confirm whether the merchant is on the legacy SOAP integration or the new REST integration before troubleshooting
DHL India — REST not yet implemented	If the merchant's account is DHL India, REST is not available; SOAP path applies
Two credential sets required	Primary: `username` + `password` (Basic Auth set 1); Secondary: `username` + `password` (Basic Auth set 2) — confirm both are entered correctly in carrier setup
Two account identifiers required	`shipperAccountNumber` and `accountNumber` — confirm both are configured in hamburger menu > Carriers
Sandbox vs. live environment	Confirm which environment credentials are active; sandbox labels must not route to production
Saturday Delivery special service	Label generation will fail if the DHL account does not support Saturday Delivery — this is an account-level limitation, not a bug
COD not supported (expected behaviour)	COD is not supported for DHL Express REST — this is expected behaviour. The COD request node should not be passed in the API request; confirm it is removed
Service Point not implemented	Not available in either SOAP or REST at this time
Add Buffer Days to Estimated Delivery	REST-only feature; not available in SOAP — confirm carrier type before troubleshooting
Special Services in Automation Rules / Bulk Actions	Implemented and working in both REST and SOAP
Edit Package flow	The new Edit Package flow should be enabled and used for DHL Express REST
PLT, Archive Airway Bill, DHL Email Service	All working as expected — settings save and persist correctly

Toggle / config note	What support should confirm
No Signature service code	Fixed — correctly maps to SX (previously mis-mapped to SA)
Platforms in scope	Shopify, WooCommerce, BigCommerce, Magento, PrestaShop

---

Step-by-Step Support Walkthrough

1. Confirm Carrier Registration and Credential Setup

1. Navigate to **hamburger menu > Carriers** in the MCSL app on the merchant's platform (Shopify Admin > Apps > PH Multi Carrier Shipping Label App; WooCommerce admin > PluginHive MCSL settings; BigCommerce MCSL app settings; Magento admin > PluginHive Multi Carrier Shipping Label extension; PrestaShop admin > PluginHive Multi Carrier Shipping Label module).
2. Confirm DHL Express is registered under Carrier Code **C42** (REST integration).
3. Confirm both credential sets are entered:

› Basic Auth Set 1: `setusername` / `password`

› Basic Auth Set 2: `username` / `password`

4. Confirm both account identifiers are present:

› `shipperAccountNumber`

› `accountNumber`

5. Confirm whether the carrier is set to sandbox or live/production mode.
6. Verify the carrier connects successfully after saving credentials.

› **Request nodes to verify:** Authentication headers (both Basic Auth sets), `shipperAccountNumber`, `accountNumber` in the outbound carrier registration request log.

---

2. Verify Carrier Services in Rate Automation and Label Automation

1. Navigate to **hamburger menu > Settings > Shipping Rates / Automation** (or Rate Automation, depending on platform).
2. Confirm all expected DHL Express services are listed and selectable under Rate Automation.
3. Navigate to Label Automation and confirm the same services appear.
4. Navigate to **hamburger menu > Carriers > Carrier Services Settings** for DHL Express (C42) and confirm the service list matches what is shown in automation.

---

3. Verify Rates at Checkout

1. Place a test order on the merchant's platform:

› **Shopify:** Shopify Admin > Orders, or trigger a checkout on the storefront.

› **WooCommerce:** WooCommerce Orders or storefront checkout.

› **BigCommerce:** BigCommerce storefront checkout or rate automation logs.

› **Magento:** Magento checkout or shipping logs.

› **PrestaShop:** PrestaShop checkout or shipping logs.

2. Confirm DHL Express rates appear with correct service name, price, and transit time.
3. Test with a multi-package order and confirm the total rate accumulates correctly across all packages.
4. If a specific service is pre-selected in carrier settings, confirm only that service appears at checkout.

› **Request/response nodes to verify:** Service name, rate amount, transit time, currency in the rate response payload. If rates are not returned, check for `"status": "404"` / `"detail": "410304: No products available"` in the rate response — this indicates no eligible products for the account/service combination.

---

4. Verify Shipping Label Generation

1. Open a test order in the **ORDERS tab**, then select **Prepare Shipment / Generate Label**.
2. Test a domestic order — confirm the label generates and downloads successfully.
3. Test an international order — confirm both the shipping label and commercial invoice are included in the output.
4. Confirm the tracking number is saved to the order after label creation.
5. If label size is configured (standard, A4, ZPL), confirm the correct format is produced.
6. If Air Waybill is enabled in carrier settings (**hamburger menu > Carriers > Other Details**), confirm it is included alongside the label.
7. If a company logo is configured in Other Details, confirm it appears on the label.

› **Request/log fields to verify:** Label format field, Air Waybill flag, tracking number in the label response, company logo reference.

---

5. Verify Return Label Generation

1. From the **ORDERS tab**, open a completed shipment and trigger return label generation.
2. Confirm the ship-from and ship-to addresses are correctly swapped on the return label.
3. Confirm the return tracking number is saved to the order.

› **Request nodes to verify:** `shipper` and `recipient` address blocks in the return label request — confirm they are reversed relative to the outbound label request.

---

6. Verify Carrier Other Details Configuration

1. Navigate to **hamburger menu > Carriers**, open DHL Express (C42), and go to **Other Details**.
2. Confirm the following settings are available and save correctly:

› Pickup Special Services (Saturday Pickup, Holiday Pickup)

› Pickup Time / Start Time / Company Close Time

› Invoice Type (Commercial Invoice), Reason for Export, Trader Type

› Package Dimensions

› Paperless Trade (PLT)

› Archive Airway Bill

› DHL Email Service

› Label Format / Label Size

› Account Rates

› DHL Invoice, Duty Payment Type, Invoice Language

› Rate Cost (with tax and without tax)

› Terms of Trade (DTP) as a separate field

› Number of Airway Bills

› Custom Shipment Message

› Include Shipping Charges in Invoice / Include Insurance Charges in Invoice

› Company Logo Upload

› Custom Reference on Label / Choose Reference to Display on Label

› `placeOfIncoterm`

3. **Working as expected:** PLT, Archive Airway Bill, and DHL Email Service save correctly and persist the selection.

---

7. Verify Special Services

1. From the **ORDERS tab**, open an order and navigate to **Prepare Shipment / Generate Label** or the order grid page.
2. Test the following special services:

Special Service	Expected behaviour	Known issue
Saturday Delivery	Node passes in request; label generation will fail if account does not support it	Account-level limitation — not a bug
Direct Signature	Applies correctly to shipment request	None detected

Special Service	Expected behaviour	Known issue
No Signature Required	Must pass `serviceCode: SX`	**Known issue:** Incorrectly passes SA — causes label failure
Paperless Trade (PLT)	Applied to shipment request	Save persistence issue (see Step 6)
Shippers Reference	Included in request	None detected
Filing Types for Shipments	Included in request	None detected
Unit of Measure	Included in request	None detected
Registration Number Type for Recipient	Included in request	None detected

Special Service	Expected behaviour	Known issue
Dangerous Goods (DG)	Flagged on shipment request	None detected
COD	**Not supported** — should not appear or be passed	COD request node must be removed from API request

3. Confirm Special Services are **not** available in Automation Rules or Bulk Actions for DHL Express REST —

WSS: Add flat rate adjustment shown when carrier rates fail (ZD #394137) - WSS: Add flat rate adjustment shown when carrier rates fail (ZD #394137)

Support Guide: WSS — Flat Rate Adjustment Suppressed When Carrier Rates Fail

WSS: Add flat rate adjustment shown when carrier rates fail (ZD #394137)

Feature Summary

This fix addresses a bug in WooCommerce Shipping Services (MCSL) where a **Rate Automation rule using the "Add Flat Rate" action** would incorrectly display the flat rate adjustment as a standalone shipping option at checkout when the configured carrier (e.g., UPS) failed to return rates. Instead of suppressing all rates on carrier failure, the adjustment value was surfaced alone — causing merchants' customers to be undercharged by paying only the small adjustment amount rather than the full carrier rate plus adjustment. The fix ensures that when a carrier is configured but returns no services (API failure), the flat rate adjustment is silently suppressed. Flat-rate-only rules (no carrier attached) continue to emit the standalone flat rate as expected.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggles detected	No gating required; fix is applied at the automation logic layer
Rate Automation rule with "Add Flat Rate" action	Confirm the merchant has at least one active automation rule using the **Add Flat Rate** or **Add Flat Rate (Quantity-based)** action
Carrier configured in the rule	Confirm a carrier (e.g., UPS) is attached to the automation rule — this distinguishes a carrier-backed rule from a flat-rate-only rule
UPS carrier account connected	Confirm UPS credentials are saved and active under hamburger menu > Carriers; the bug is triggered when UPS (or any carrier) fails to return rates

Toggle / config note	What support should confirm
WooCommerce store on MCSL SL 381 or later	Confirm the store is on the release that includes this fix; confirm from live store/card context if exact build version is visible
Live carrier API reachability	Confirm whether UPS API failures are intermittent or consistent — the bug only manifests when the carrier call fails

---

Step-by-Step Support Walkthrough

1. **Confirm the automation rule configuration.**

Navigate to **hamburger menu > Settings > Shipping Rates / Automation** (or the Rate Automation section within the MCSL app on the WooCommerce store admin).

› Locate the rule the merchant reported.

› Confirm the rule action is set to **Add Flat Rate** or **Add Flat Rate (Quantity-based)**.

› Confirm a carrier (e.g., UPS) is selected/attached to the rule.

› Note the flat rate adjustment value configured.

2. **Confirm the UPS carrier account status.**

Navigate to **hamburger menu > Carriers**.

› Verify UPS credentials (account number, API keys) are saved and active.

› Check whether the carrier shows any connectivity warning or error state.

3. **Reproduce or identify the failure condition.**

Place a test order or review the reported order in **WooCommerce Orders** (WordPress Admin > WooCommerce > Orders).

› Identify the order where the incorrect flat rate appeared at checkout.

› Note the shipping amount charged — if it matches only the flat rate adjustment value (not a full carrier rate), the bug was active.

4. **Inspect the request log for the carrier API failure.**

Navigate to **hamburger menu > Settings > Request Log** (or the equivalent log viewer in MCSL).

› Filter by the affected order or the relevant date/time window.

› Look for the UPS rate request associated with the checkout session.

› Confirm the UPS API returned an error or empty services response.

› `- Request/log fields to verify: carrierId` presence in the automation action object; `services` array being empty or absent in the UPS rate response; the flat rate value that was incorrectly emitted as a standalone rate.

5. **Verify post-fix behaviour on a test checkout.**

› With the fix applied, simulate a checkout where UPS rates are unavailable (e.g., temporarily use invalid credentials or an unsupported destination to force a carrier failure).

› Confirm that **no shipping options appear at checkout** when UPS fails — the flat rate adjustment should not surface as a standalone option.

› Restore valid credentials and confirm that when UPS returns rates successfully, the flat rate adjustment is correctly added on top of the carrier rate.

6. **Verify flat-rate-only rules are unaffected.**

› Locate or create an automation rule that uses **Add Flat Rate** with **no carrier attached** (flat-rate-only rule).

› Confirm this rule still emits the standalone flat rate at checkout as expected — this is the intended behaviour for carrier-free rules.

Expected Behaviour

What support should observe	How to confirm
When UPS (or any carrier) fails to return rates, **no shipping options appear at checkout**	Test checkout with a forced carrier failure; confirm the checkout shipping section is empty or shows only non-carrier-backed options
The flat rate adjustment value is **not shown as a standalone rate** when the carrier is configured but fails	Confirm the checkout total does not reflect only the adjustment amount
When UPS returns rates successfully, the flat rate adjustment is **added on top of the carrier rate**	Test checkout with valid UPS credentials and a supported destination; confirm rate = carrier rate + adjustment
Flat-rate-only automation rules (no carrier attached) **continue to emit the standalone flat rate**	Test a rule with no carrier configured; confirm the flat rate appears at checkout as before
Request log shows `carrierId` is set and `services` is empty when the carrier fails	Review the log entry for the failed UPS call; `carrierId` present + empty `services` = suppression triggered
Request log shows `carrierId` is absent for flat-rate-only rules	Review the log entry for a flat-rate-only rule; no `carrierId` = standalone flat rate correctly emitted

---

Merchant-Safe Explanation

Previously, if your UPS shipping rates couldn't be retrieved at checkout — due to a temporary API issue or configuration problem — our system was incorrectly showing the flat rate adjustment you had configured as a standalone shipping option. This meant your customers could check out paying only that small adjustment amount instead of the full shipping cost. This has now been fixed: if UPS fails to return rates, no shipping option will be shown at checkout for that rule, preventing undercharging. If you have a shipping rule that uses a flat rate with no carrier attached, that will continue to work exactly as before.

Common Questions & Troubleshooting

› **Check first:** Confirm the merchant's automation rule has a carrier (UPS) attached. If no carrier is attached, the standalone flat rate appearing at checkout is correct behaviour — not a bug.

› **Check first:** Confirm the store is running the MCSL build that includes this fix (SL 381 or later). If the store is on an older build, the bug may still be present regardless of configuration.

› **If the flat rate still appears after the fix:** Inspect the request log to confirm whether `carrierId` is being set on the automation action object. If `carrierId` is missing even though a carrier is configured in the rule, escalate — the rule may not be saving the carrier association correctly.

› **If UPS failures are frequent:** Investigate the root cause of the carrier API failure separately (credentials, account status, destination restrictions). The fix suppresses the incorrect rate but does not resolve the underlying carrier connectivity issue.

› **If the merchant reports customers were undercharged historically:** This is a known consequence of the bug. Confirm the affected orders and escalate to the account manager if refund or billing remediation is needed — do not attempt to retroactively adjust orders from the MCSL side.

› **Inspect logs when:** The merchant reports a flat rate appearing at checkout unexpectedly, or when no rates appear and the merchant expected carrier rates. The request log will show whether the carrier call succeeded or failed and what the automation layer emitted.

› **Escalate when:** The fix is confirmed deployed but the incorrect flat rate still surfaces; `carrierId` is not being set correctly on rules with a carrier configured; or the merchant needs historical order impact assessment.

Support Escalation Packet

› **Story card:** WSS: Add flat rate adjustment shown when carrier rates fail (ZD #394137)

Trello URL: https://trello.com/c/NLqXsEv7/4575-wss-add-flat-rate-adjustment-shown-when-carrier-rates-fail-zd-394137

› **ZD ticket:** https://pluginhive.zendesk.com/agent/tickets/394137

› **Store URL:** *(Collect from merchant/ticket)*

› **Account UUID:** *(Collect from MCSL account record)*

› **Carrier account:** UPS — account number and API credential status *(collect from merchant or hamburger menu > Carriers)*

› **Affected order number(s):** *(Collect from WooCommerce Orders — orders where only the flat rate adjustment amount was charged)*

› **Generated label / tracking identifier:** *(Collect if a label was generated on the undercharged order)*

› **Screenshots required:**

› Automation rule setup showing the Add Flat Rate action and carrier selection (hamburger menu > Settings > Rate Automation)

› Checkout screenshot or order record showing the incorrect flat rate amount

› Request log entry showing the UPS API failure (`services` empty, `carrierId` present) and the flat rate value that was emitted

› Post-fix checkout screenshot confirming no rate appears when carrier fails

› **Exact toggle/config value:** No toggle gating — confirm MCSL build version is SL 381 or later and include the build/version string in the escalation packet